

17 Kernel Methods *

In this chapter, we consider **nonparametric methods** for regression and classification. Such methods do not assume a fixed parametric form for the prediction function, but instead try to estimate the function itself (rather than the parameters) directly from data. The key idea is that we observe the function value at a fixed set of N points, namely $y_n = f(\mathbf{x}_n)$ for $n = 1 : N$, where f is the unknown function, so to predict the function value at a new point, say $\mathbf{x}_*$, we just have to compare how “similar” $\mathbf{x}_*$ is to each of the N training points, $\{\mathbf{x}_n\}$, and then we can predict that $f(\mathbf{x}_*)$ is some weighted combination of the $\{f(\mathbf{x}_n)\}$ values. Thus we may need to “remember” the entire training set, $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}$, in order to make predictions at test time — we cannot “compress” $\mathcal{D}$ into a fixed-sized parameter vector.

The weights that are used for prediction are determined by the similarity between $\mathbf{x}_*$ and each $\mathbf{x}_n$, which is computed using a special kind of function known as kernel function, $\mathcal{K}(\mathbf{x}_n, \mathbf{x}_*) \geq 0$, which we explain in Section 17.1. This approach is similar to RBF networks (Section 13.6.1), except we use the datapoints $\{\mathbf{x}_n\}$ themselves as the “anchors”, rather than learning the RBF centroids $\{\boldsymbol{\mu}_k\}$.

In Section 17.2, we discuss an approach called Gaussian processes, which allows us to use the kernel to define a *prior over functions*, which we can update given data to get a *posterior over functions*. Alternatively we can use the same kernel with a method called Support Vector Machines to compute a MAP estimate of the function, as we explain in Section 17.3.

17.1 Mercer kernels

The key to nonparametric methods is that we need a way to encode prior knowledge about the similarity of two input vectors. If we know that $\mathbf{x}_i$ is similar to $\mathbf{x}_j$, then we can encourage the model to make the predicted output at both locations (i.e., $f(\mathbf{x}_i)$ and $f(\mathbf{x}_j)$) to be similar.

To define similarity, we introduce the notion of a **kernel function**. The word “kernel” has many different meanings in mathematics, including density kernels (Section 16.3.1), transition kernels of a Markov chain (Section 3.6.1.2), and convolutional kernels (Section 14.1). Here we consider a **Mercer kernel**, also called a **positive definite kernel**. This is any symmetric function $\mathcal{K} : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$ such that

$$\sum_{i=1}^N \sum_{j=1}^N \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) c_i c_j \geq 0 \quad (17.1)$$

for any set of N (unique) points $\mathbf{x}_i \in \mathcal{X}$, and any choice of numbers $c_i \in \mathbb{R}$. (We assume $\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) > 0$, so that we can only achieve equality in the above equation if $c_i = 0$ for all i .)

Another way to understand this condition is the following. Given a set of N datapoints, let us define the **Gram matrix** as the following $N \times N$ similarity matrix:

$$\mathbf{K} = \begin{pmatrix} \mathcal{K}(\mathbf{x}_1, \mathbf{x}_1) & \cdots & \mathcal{K}(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & & \vdots \\ \mathcal{K}(\mathbf{x}_N, \mathbf{x}_1) & \cdots & \mathcal{K}(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix} \quad (17.2)$$

We say that $\mathcal{K}$ is a Mercer kernel iff the Gram matrix is positive definite for any set of (distinct) inputs $\{\mathbf{x}_i\}_{i=1}^N$.

The most widely used kernel for real-valued inputs is the **squared exponential kernel** (SE), also called the **exponentiated quadratic kernel** (EQ), **Gaussian kernel**, or **RBF kernel**. It is defined by

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) \quad (17.3)$$

Here ℓ corresponds to the length scale of the kernel, i.e., the distance over which we expect differences to matter. This is known as the **bandwidth** parameter. The RBF kernel measures similarity between two vectors in $\mathbb{R}^D$ using (scaled) Euclidean distance. In Section 17.1.2, we will discuss several other kinds of kernel.

In Section 17.2, we show how to use kernels to define priors and posteriors over functions. The basic idea is this: if $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ is large, meaning the inputs are similar, then we expect the output of the function to be similar as well, so $f(\mathbf{x}) \approx f(\mathbf{x}')$. More precisely, information we learn about $f(\mathbf{x})$ will help us predict $f(\mathbf{x}')$ for all $\mathbf{x}'$ which are correlated with $\mathbf{x}$, and hence for which $\mathcal{K}(\mathbf{x}, \mathbf{x}')$ is large.

In Section 17.3, we show how to use kernels to generalize from Euclidean distance to a more general notion of distance, so that we can use geometric methods such as linear discriminant analysis in an implicit feature space instead of input space.

17.1.1 Mercer's theorem

Recall from Section 7.4 that any positive definite matrix $\mathbf{K}$ can be represented using an eigendecomposition of the form $\mathbf{K} = \mathbf{U}^\top \mathbf{\Lambda} \mathbf{U}$, where $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues $\lambda_i > 0$, and $\mathbf{U}$ is a matrix containing the eigenvectors. Now consider element (i, j) of $\mathbf{K}$:

$$k_{ij} = (\mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:i})^\top (\mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:j}) \quad (17.4)$$

where $\mathbf{U}_{:i}$ is the i 'th column of $\mathbf{U}$. If we define $\phi(\mathbf{x}_i) = \mathbf{\Lambda}^{\frac{1}{2}} \mathbf{U}_{:i}$, then we can write

$$k_{ij} = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) = \sum_m \phi_m(\mathbf{x}_i) \phi_m(\mathbf{x}_j) \quad (17.5)$$

Thus we see that the entries in the kernel matrix can be computed by performing an inner product of some feature vectors that are implicitly defined by the eigenvectors of the kernel matrix. This idea can be generalized to apply to kernel functions, not just kernel matrices; this result is known as **Mercer's theorem**.

For example, consider the **quadratic kernel** $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \langle \mathbf{x}, \mathbf{x}' \rangle^2$. In 2d, we have

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = (x_1 x'_1 + x_2 x'_2)^2 = x_1^2 (x'_1)^2 + 2x_1 x_2 x'_1 x'_2 + x_2^2 (x'_2)^2 \quad (17.6)$$

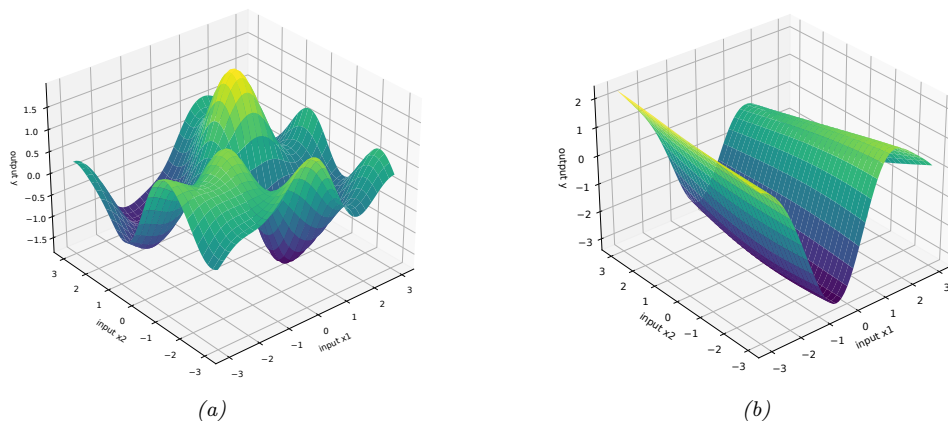


Figure 17.1: Function samples from a GP with an ARD kernel. (a) $\ell_1 = \ell_2 = 1$. Both dimensions contribute to the response. (b) $\ell_1 = 1, \ell_2 = 5$. The second dimension is essentially ignored. Adapted from Figure 5.1 of [RW06]. Generated by [gprDemoArd.ipynb](#).

We can write this as $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$ if we define $\phi(x_1, x_2) = [x_1^2, \sqrt{2}x_1x_2, x_2^2] \in \mathbb{R}^3$. So we embed the 2d inputs $\mathbf{x}$ into a 3d feature space $\phi(\mathbf{x})$.

Now consider the RBF kernel. In this case, the corresponding feature representation is infinite dimensional (see Section 17.2.9.3 for details). However, by working with kernel functions, we can avoid having to deal with infinite dimensional vectors.

17.1.2 Some popular Mercer kernels

In the sections below, we describe some popular Mercer kernels. More details can be found at [Wil14] and <https://www.cs.toronto.edu/~duvenaud/cookbook/>.

17.1.2.1 Stationary kernels for real-valued vectors

For real-valued inputs, $\mathcal{X} = \mathbb{R}^D$, it is common to use **stationary kernels**, which are functions of the form $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}(\|\mathbf{x} - \mathbf{x}'\|)$; thus the value only depends on the elementwise difference between the inputs. The RBF kernel is a stationary kernel. We give some other examples below.

ARD kernel

We can generalize the RBF kernel by replacing Euclidean distance with Mahalanobis distance, as follows:

$$\mathcal{K}(\mathbf{r}) = \sigma^2 \exp\left(-\frac{1}{2}\mathbf{r}^\top \Sigma^{-1}\mathbf{r}\right) \quad (17.7)$$



Figure 17.2: Functions sampled from a GP with a Matern kernel. (a) $\nu = 5/2$. (b) $\nu = 1/2$. Generated by `gpKernelPlot.ipynb`.

where $\mathbf{r} = \mathbf{x} - \mathbf{x}'$. If Σ is diagonal, this can be written as

$$\mathcal{K}(\mathbf{r}; \ell, \sigma^2) = \sigma^2 \exp \left(-\frac{1}{2} \sum_{d=1}^D \frac{1}{\ell_d^2} r_d^2 \right) = \prod_{d=1}^D \mathcal{K}(r_d; \ell_d, \sigma^{2/D}) \quad (17.8)$$

where

$$\mathcal{K}(r; \ell, \tau^2) = \tau^2 \exp \left(-\frac{1}{2} \frac{1}{\ell^2} r^2 \right) \quad (17.9)$$

We can interpret σ^2 as the overall variance, and ℓ_d as defining the **characteristic length scale** of dimension d . If d is an irrelevant input dimension, we can set $\ell_d = \infty$, so the corresponding dimension will be ignored. This is known as **automatic relevancy determination** or **ARD** (Section 11.7.7). Hence the corresponding kernel is called the **ARD kernel**. See Figure 17.1 for an illustration of some 2d functions sampled from a GP using this prior.

Matern kernels

The SE kernel gives rise to functions that are infinitely differentiable, and therefore are very smooth. For many applications, it is better to use the **Matern kernel**, which gives rise to “rougher” functions, which can better model local “wiggles” without having to make the overall length scale very small.

The Matern kernel has the following form:

$$\mathcal{K}(r; \nu, \ell) = \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu} r}{\ell} \right)^\nu K_\nu \left(\frac{\sqrt{2\nu} r}{\ell} \right) \quad (17.10)$$

where K_ν is a modified Bessel function and ℓ is the length scale. Functions sampled from this GP are k -times differentiable iff $\nu > k$. As $\nu \rightarrow \infty$, this approaches the SE kernel.



Figure 17.3: Functions sampled from a GP using various stationary periodic kernels. Generated by [gpKernelPlot.ipynb](#).

For values $\nu \in \{\frac{1}{2}, \frac{3}{2}, \frac{5}{2}\}$, the function simplifies as follows:

$$\mathcal{K}(r; \frac{1}{2}, \ell) = \exp\left(-\frac{r}{\ell}\right) \quad (17.11)$$

$$\mathcal{K}(r; \frac{3}{2}, \ell) = \left(1 + \frac{\sqrt{3}r}{\ell}\right) \exp\left(-\frac{\sqrt{3}r}{\ell}\right) \quad (17.12)$$

$$\mathcal{K}(r; \frac{5}{2}, \ell) = \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right) \quad (17.13)$$

The value $\nu = \frac{1}{2}$ corresponds to the **Ornstein-Uhlenbeck process**, which describes the velocity of a particle undergoing Brownian motion. The corresponding function is continuous but not differentiable, and hence is very “jagged”. See Figure 17.2b for an illustration.

Periodic kernels

The **periodic kernel** captures repeating structure, and has the form

$$\mathcal{K}_{\text{per}}(r; \ell, p) = \exp\left(-\frac{2}{\ell^2} \sin^2\left(\pi \frac{r}{p}\right)\right) \quad (17.14)$$

where p is the period. See Figure 17.3a for an illustration.

A related kernel is the **cosine kernel**:

$$\mathcal{K}(r; p) = \cos\left(2\pi \frac{r}{p}\right) \quad (17.15)$$

See Figure 17.3b for an illustration.

17.1.2.2 Making new kernels from old

Given two valid kernels $\mathcal{K}_1(\mathbf{x}, \mathbf{x}')$ and $\mathcal{K}_2(\mathbf{x}, \mathbf{x}')$, we can create a new kernel using any of the following methods:

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = c\mathcal{K}_1(\mathbf{x}, \mathbf{x}'), \text{ for any constant } c > 0 \quad (17.16)$$

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})\mathcal{K}_1(\mathbf{x}, \mathbf{x}')f(\mathbf{x}'), \text{ for any function } f \quad (17.17)$$

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = q(\mathcal{K}_1(\mathbf{x}, \mathbf{x}')) \text{ for any function polynomial } q \text{ with nonneg. coef.} \quad (17.18)$$

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp(\mathcal{K}_1(\mathbf{x}, \mathbf{x}')) \quad (17.19)$$

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{A} \mathbf{x}', \text{ for any psd matrix } \mathbf{A} \quad (17.20)$$

For example, suppose we start with the linear kernel $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$. We know this is a valid Mercer kernel, since the corresponding Gram matrix is just the (scaled) covariance matrix of the data. From the above rules, we can see that the polynomial kernel $\mathcal{K}(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}')^M$ is a valid Mercer kernel. This contains all monomials of order M . For example, if $M = 2$ and the inputs are 2d, we have

$$(\mathbf{x}^\top \mathbf{x}')^2 = (x_1x'_1 + x_2x'_2)^2 = (x_1x'_1)^2 + (x_2x'_2)^2 + 2(x_1x'_1)(x_2x'_2) \quad (17.21)$$

We can generalize this to contain all terms up to degree M by using the kernel $\mathcal{K}(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^M$. For example, if $M = 2$ and the inputs are 2d, we have

$$\begin{aligned} (\mathbf{x}^\top \mathbf{x}' + 1)^2 &= (x_1x'_1)^2 + (x_1x'_1)(x_2x'_2) + (x_1x'_1) \\ &\quad + (x_2x'_2)(x_1x'_1) + (x_2x'_2)^2 + (x_2x'_2) \\ &\quad + (x_1x'_1) + (x_2x'_2) + 1 \end{aligned} \quad (17.22)$$

We can also use the above rules to establish that the Gaussian kernel is a valid kernel. To see this, note that

$$\|\mathbf{x} - \mathbf{x}'\|^2 = \mathbf{x}^\top \mathbf{x} + (\mathbf{x}')^\top \mathbf{x}' - 2\mathbf{x}^\top \mathbf{x}' \quad (17.23)$$

and hence

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2/2\sigma^2) = \exp(-\mathbf{x}^\top \mathbf{x}/2\sigma^2) \exp(\mathbf{x}^\top \mathbf{x}'/\sigma^2) \exp(-(\mathbf{x}')^\top \mathbf{x}'/2\sigma^2) \quad (17.24)$$

is a valid kernel.

17.1.2.3 Combining kernels by addition and multiplication

We can also combine kernels using addition or multiplication:

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}_1(\mathbf{x}, \mathbf{x}') + \mathcal{K}_2(\mathbf{x}, \mathbf{x}') \quad (17.25)$$

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}_1(\mathbf{x}, \mathbf{x}') \times \mathcal{K}_2(\mathbf{x}, \mathbf{x}') \quad (17.26)$$

Multiplying two positive-definite kernels together always results in another positive definite kernel. This is a way to get a conjunction of the individual properties of each kernel, as illustrated in Figure 17.4.

In addition, adding two positive-definite kernels together always results in another positive definite kernel. This is a way to get a disjunction of the individual properties of each kernel, as illustrated in Figure 17.5.

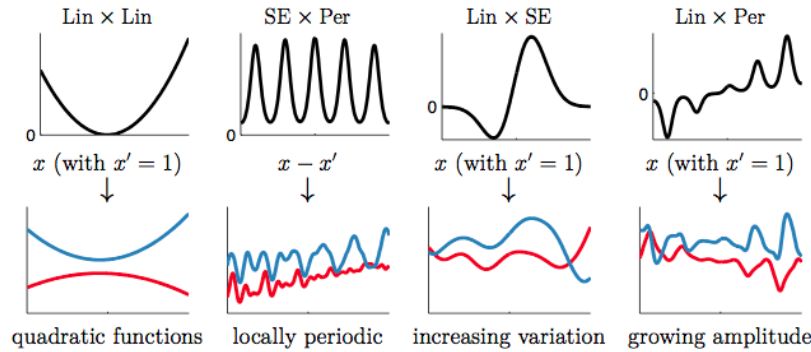


Figure 17.4: Examples of 1d structures obtained by multiplying elementary kernels. Top row shows $K(x, x' = 1)$. Bottom row shows some functions sampled from $GP(f|0, K)$. From Figure 2.2 of [Duv14]. Used with kind permission of David Duvenaud.

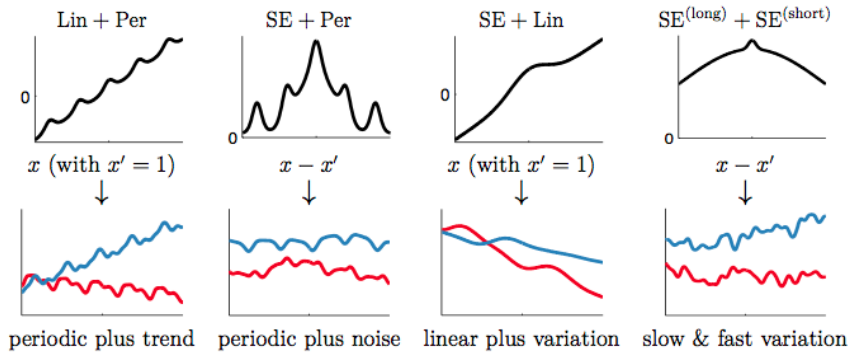


Figure 17.5: Examples of 1d structures obtained by adding elementary kernels. Here $SE^{(short)}$ and $SE^{(long)}$ are two SE kernels with different length scales. From Figure 2.4 of [Duv14]. Used with kind permission of David Duvenaud.

17.1.2.4 Kernels for structured inputs

Kernels are particularly useful when the inputs are structured objects, such as strings and graphs, since it is often hard to “featurize” variable-sized inputs. For example, we can define a **string kernel** which compares strings in terms of the number of n -grams they have in common [Lod+02; BC17].

We can also define kernels on graphs [KJM19]. For example, the **random walk kernel** conceptually performs random walks on two graphs simultaneously, and then counts the number of paths that were produced by both walks. This can be computed efficiently as discussed in [Vis+10]. For more details on graph kernels, see [KJM19].

For a review of kernels on structured objects, see e.g., [Gär03].

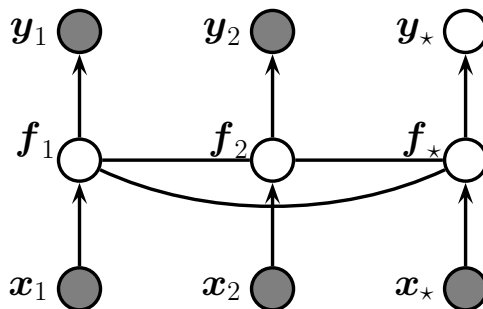


Figure 17.6: A Gaussian process for 2 training points, $\mathbf{x}_1$ and $\mathbf{x}_2$, and 1 testing point, $\mathbf{x}_*$, represented as a graphical model representing $p(\mathbf{y}, \mathbf{f}_X | \mathbf{X}) = \mathcal{N}(\mathbf{f}_X | m(\mathbf{X}), \mathcal{K}(\mathbf{X})) \prod_i p(y_i | f_i)$. The hidden nodes $f_i = f(\mathbf{x}_i)$ represent the value of the function at each of the data points. These hidden nodes are fully interconnected by undirected edges, forming a Gaussian graphical model; the edge strengths represent the covariance terms $\Sigma_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$. If the test point $\mathbf{x}_*$ is similar to the training points $\mathbf{x}_1$ and $\mathbf{x}_2$, then the value of the hidden function f_* will be similar to f_1 and f_2 , and hence the predicted output y_* will be similar to the training values y_1 and y_2 .

17.2 Gaussian processes

In this section, we discuss **Gaussian processes**, which is a way to define distributions over functions of the form $f : \mathcal{X} \rightarrow \mathbb{R}$, where $\mathcal{X}$ is any domain. The key assumption is that the function values at a set of $M > 0$ inputs, $\mathbf{f} = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_M)]$, is jointly Gaussian, with mean ($\boldsymbol{\mu} = m(\mathbf{x}_1), \dots, m(\mathbf{x}_M)$) and covariance $\Sigma_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$, where m is a mean function and $\mathcal{K}$ is a positive definite (Mercer) kernel. Since we assume this holds for any $M > 0$, this includes the case where $M = N + 1$, containing N training points $\mathbf{x}_n$ and 1 test point $\mathbf{x}_*$. Thus we can infer $f(\mathbf{x}_*)$ from knowledge of $f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)$ by manipulating the joint Gaussian distribution $p(f(\mathbf{x}_1), \dots, f(\mathbf{x}_N), f(\mathbf{x}_*))$, as we explain below. We can also extend this to work with the case where we observe noisy functions of $f(\mathbf{x}_n)$, such as in regression or classification problems.

17.2.1 Noise-free observations

Suppose we observe a training set $\mathcal{D} = \{(\mathbf{x}_n, y_n) : n = 1 : N\}$, where $y_n = f(\mathbf{x}_n)$ is the noise-free observation of the function evaluated at $\mathbf{x}_n$. If we ask the GP to predict $f(\mathbf{x})$ for a value of $\mathbf{x}$ that it has already seen, we want the GP to return the answer $f(\mathbf{x})$ with no uncertainty. In other words, it should act as an **interpolator** of the training data.

Now we consider the case of predicting the outputs for new inputs that may not be in $\mathcal{D}$. Specifically, given a test set $\mathbf{X}_*$ of size $N_* \times D$, we want to predict the function outputs $\mathbf{f}_* = [f(\mathbf{x}_{*1}), \dots, f(\mathbf{x}_{*N_*})]$. By definition of the GP, the joint distribution $p(\mathbf{f}_X, \mathbf{f}_* | \mathbf{X}, \mathbf{X}_*)$ has the following form

$$\begin{pmatrix} \mathbf{f}_X \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N} \left(\begin{pmatrix} \boldsymbol{\mu}_X \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K}_{X,X} & \mathbf{K}_{X,*} \\ \mathbf{K}_{X,*}^\top & \mathbf{K}_{*,*} \end{pmatrix} \right) \quad (17.27)$$

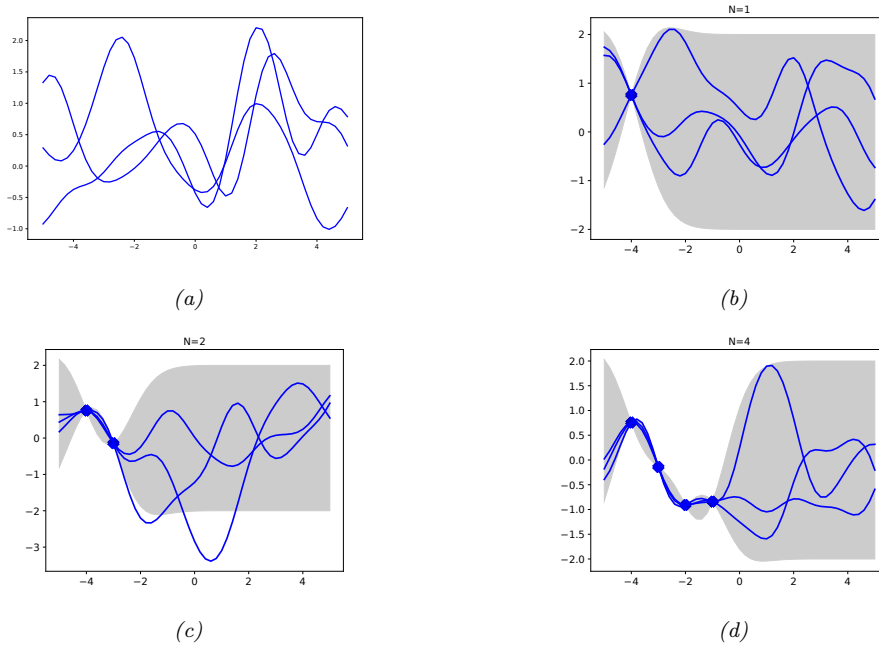


Figure 17.7: (a) some functions sampled from a GP prior with squared exponential kernel. (b-d) : some samples from a GP posterior, after conditioning on 1,2, and 4 noise-free observations. The shaded area represents $\mathbb{E}[f(\mathbf{x})] \pm 2\text{std}[f(\mathbf{x})]$. Adapted from Figure 2.2 of [RW06]. Generated by [gprDemoNoiseFree.ipynb](#).

where $\boldsymbol{\mu}_X = [m(\mathbf{x}_1), \dots, m(\mathbf{x}_N)]$, $\boldsymbol{\mu}_* = [m(\mathbf{x}_1^*), \dots, m(\mathbf{x}_{N_*}^*)]$, $\mathbf{K}_{X,X} = \mathcal{K}(\mathbf{X}, \mathbf{X})$ is $N \times N$, $\mathbf{K}_{X,*} = \mathcal{K}(\mathbf{X}, \mathbf{X}_*)$ is $N \times N_*$, and $\mathbf{K}_{*,*} = \mathcal{K}(\mathbf{X}_*, \mathbf{X}_*)$ is $N_* \times N_*$. See Figure 17.6 for an illustration. By the standard rules for conditioning Gaussians (Section 3.2.3), the posterior has the following form

$$p(\mathbf{f}_* | \mathbf{X}_*, \mathcal{D}) = \mathcal{N}(\mathbf{f}_* | \boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*) \quad (17.28)$$

$$\boldsymbol{\mu}_* = m(\mathbf{X}_*) + \mathbf{K}_{X,*,*}^T \mathbf{K}_{X,X}^{-1} (\mathbf{f}_X - m(\mathbf{X})) \quad (17.29)$$

$$\boldsymbol{\Sigma}_* = \mathbf{K}_{*,*} - \mathbf{K}_{X,*,*}^T \mathbf{K}_{X,X}^{-1} \mathbf{K}_{X,*} \quad (17.30)$$

This process is illustrated in Figure 17.7. On the left we show some samples from the prior, $p(f)$, where we use an RBF kernel (Section 17.1) and a zero mean function. On the right, we show samples from the posterior, $p(f|\mathcal{D})$. We see that the model perfectly interpolates the training data, and that the predictive uncertainty increases as we move further away from the observed data.

17.2.2 Noisy observations

Now let us consider the case where what we observe is a noisy version of the underlying function, $y_n = f(\mathbf{x}_n) + \epsilon_n$, where $\epsilon_n \sim \mathcal{N}(0, \sigma_y^2)$. In this case, the model is not required to interpolate the data,

but it must come “close” to the observed data. The covariance of the observed noisy responses is

$$\text{Cov}[y_i, y_j] = \text{Cov}[f_i, f_j] + \text{Cov}[\epsilon_i, \epsilon_j] = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) + \sigma_y^2 \delta_{ij} \quad (17.31)$$

where $\delta_{ij} = \mathbb{I}(i = j)$. In other words

$$\text{Cov}[\mathbf{y}|\mathbf{X}] = \mathbf{K}_{X,X} + \sigma_y^2 \mathbf{I}_N \triangleq \mathbf{K}_\sigma \quad (17.32)$$

The joint density of the observed data and the latent, noise-free function on the test points is given by

$$\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_X \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K}_\sigma & \mathbf{K}_{X,*} \\ \mathbf{K}_{X,*}^\top & \mathbf{K}_{*,*} \end{pmatrix}\right) \quad (17.33)$$

Hence the posterior predictive density at a set of test points $\mathbf{X}_*$ is

$$p(\mathbf{f}_*|\mathcal{D}, \mathbf{X}_*) = \mathcal{N}(\mathbf{f}_*|\boldsymbol{\mu}_{*|X}, \boldsymbol{\Sigma}_{*|X}) \quad (17.34)$$

$$\boldsymbol{\mu}_{*|X} = \boldsymbol{\mu}_* + \mathbf{K}_{X,*}^\top \mathbf{K}_\sigma^{-1}(\mathbf{y} - \boldsymbol{\mu}_X) \quad (17.35)$$

$$\boldsymbol{\Sigma}_{*|X} = \mathbf{K}_{*,*} - \mathbf{K}_{X,*}^\top \mathbf{K}_\sigma^{-1} \mathbf{K}_{X,*} \quad (17.36)$$

In the case of a single test input, this simplifies as follows

$$p(f_*|\mathcal{D}, \mathbf{x}_*) = \mathcal{N}(f_*|m_* + \mathbf{k}_*^\top \mathbf{K}_\sigma^{-1}(\mathbf{y} - \boldsymbol{\mu}_X), k_{**} - \mathbf{k}_*^\top \mathbf{K}_\sigma^{-1} \mathbf{k}_*) \quad (17.37)$$

where $\mathbf{k}_* = [\mathcal{K}(\mathbf{x}_*, \mathbf{x}_1), \dots, \mathcal{K}(\mathbf{x}_*, \mathbf{x}_N)]$ and $k_{**} = \mathcal{K}(\mathbf{x}_*, \mathbf{x}_*)$. If the mean function is zero, we can write the posterior mean as follows:

$$\boldsymbol{\mu}_{*|X} = \mathbf{k}_*^\top (\mathbf{K}_\sigma^{-1} \mathbf{y}) \triangleq \mathbf{k}_*^\top \boldsymbol{\alpha} = \sum_{n=1}^N \mathcal{K}(\mathbf{x}_*, \mathbf{x}_n) \alpha_n \quad (17.38)$$

This is identical to the predictions from kernel ridge regression in Equation (17.108).

17.2.3 Comparison to kernel regression

In Section 16.3.5, we discussed kernel regression, which is a generative approach to regression in which we approximate $p(y, \mathbf{x})$ using kernel density estimation. In particular, Equation (16.39) gives us

$$\mathbb{E}[y|\mathbf{x}, \mathcal{D}] = \frac{\sum_{n=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_n) y_n}{\sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'})} = \sum_{n=1}^N y_n w_n(\mathbf{x}) \quad (17.39)$$

$$w_n(\mathbf{x}) \triangleq \frac{\mathcal{K}_h(\mathbf{x} - \mathbf{x}_n)}{\sum_{n'=1}^N \mathcal{K}_h(\mathbf{x} - \mathbf{x}_{n'})} \quad (17.40)$$

This is very similar to Equation (17.38). However, there are a few important differences. Firstly, in a GP, we use a positive definite (Mercer) kernel instead of a density kernel; Mercer kernels can be defined on structured objects, such as strings and graphs, which is harder to do for density kernels.

Second, a GP is an interpolator (at least when $\sigma^2 = 0$), so $\mathbb{E}[y|\mathbf{x}_n, \mathcal{D}] = y_n$. By contrast, kernel regression is not an interpolator (although it can be made into one by iteratively fitting the residuals, as in [KJ16]). Third, a GP is a Bayesian method, which means we can estimate hyperparameters (of the kernel) by maximizing the marginal likelihood; by contrast, in kernel regression we must use cross-validation to estimate the kernel parameters, such as the bandwidth. Fourth, computing the weights w_n for kernel regression takes $O(N)$ time, where $N = |\mathcal{D}|$, whereas computing the weights α_n for GP regression takes $O(N^3)$ time (although there are approximation methods that can reduce this to $O(NM^2)$, as we discuss in Section 17.2.9).

17.2.4 Weight space vs function space

In this section, we show how Bayesian linear regression is a special case of a GP.

Consider the linear regression model $y = f(\mathbf{x}) + \epsilon$, where $f(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})$ and $\epsilon \sim \mathcal{N}(0, \sigma_y^2)$. If we use a Gaussian prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \boldsymbol{\Sigma}_w)$, then the posterior is as follows (see Section 11.7.2 for the derivation):

$$p(\mathbf{w}|\mathcal{D}) = \mathcal{N}(\mathbf{w} | \frac{1}{\sigma_y^2} \mathbf{A}^{-1} \boldsymbol{\Phi}^\top \mathbf{y}, \mathbf{A}^{-1}) \quad (17.41)$$

where $\boldsymbol{\Phi}$ is the $N \times D$ design matrix, and

$$\mathbf{A} = \sigma_y^{-2} \boldsymbol{\Phi}^\top \boldsymbol{\Phi} + \boldsymbol{\Sigma}_w^{-1} \quad (17.42)$$

The posterior predictive distribution for $f_* = f(\mathbf{x}_*)$ is therefore

$$p(f_*|\mathcal{D}, \mathbf{x}_*) = \mathcal{N}(f_* | \frac{1}{\sigma_y^2} \boldsymbol{\phi}_*^\top \mathbf{A}^{-1} \boldsymbol{\Phi}^\top \mathbf{y}, \boldsymbol{\phi}_*^\top \mathbf{A}^{-1} \boldsymbol{\phi}_*) \quad (17.43)$$

where $\boldsymbol{\phi}_* = \boldsymbol{\phi}(\mathbf{x}_*)$. This views the problem of inference and prediction in **weight space**.

We now show that this is equivalent to the predictions made by a GP using a kernel of the form $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\Sigma}_w \boldsymbol{\phi}(\mathbf{x}')$. To see this, let $\mathbf{K} = \boldsymbol{\Phi} \boldsymbol{\Sigma}_w \boldsymbol{\Phi}^\top$, $\mathbf{k}_* = \boldsymbol{\Phi} \boldsymbol{\Sigma}_w \boldsymbol{\phi}_*$, and $k_{**} = \boldsymbol{\phi}_*^\top \boldsymbol{\Sigma}_w \boldsymbol{\phi}_*$. Using this notation, and the matrix inversion lemma, we can rewrite Equation (17.43) as follows

$$p(f_*|\mathcal{D}, \mathbf{x}_*) = \mathcal{N}(f_* | \boldsymbol{\mu}_{*|X}, \boldsymbol{\Sigma}_{*|X}) \quad (17.44)$$

$$\boldsymbol{\mu}_{*|X} = \boldsymbol{\phi}_*^\top \boldsymbol{\Sigma}_w \boldsymbol{\Phi}^\top (\mathbf{K} + \sigma_y^2 \mathbf{I})^{-1} \mathbf{y} = \mathbf{k}_*^\top \mathbf{K}_\sigma^{-1} \mathbf{y} \quad (17.45)$$

$$\boldsymbol{\Sigma}_{*|X} = \boldsymbol{\phi}_*^\top \boldsymbol{\Sigma}_w \boldsymbol{\phi}_* - \boldsymbol{\phi}_*^\top \boldsymbol{\Sigma}_w \boldsymbol{\Phi}^\top (\mathbf{K} + \sigma_y^2 \mathbf{I})^{-1} \boldsymbol{\Phi} \boldsymbol{\Sigma}_w \boldsymbol{\phi}_* = k_{**} - \mathbf{k}_*^\top \mathbf{K}_\sigma^{-1} \mathbf{k}_* \quad (17.46)$$

which matches the results in Equation (17.37), assuming $m(\mathbf{x}) = 0$. (Non-zero mean can be captured by adding a constant feature with value 1 to $\boldsymbol{\phi}(\mathbf{x})$.)

Thus we can derive a GP from Bayesian linear regression. Note, however, that linear regression assumes $\boldsymbol{\phi}(\mathbf{x})$ is a finite length vector, whereas a GP allows us to work directly in terms of kernels, which may correspond to infinite length feature vectors (see Section 17.1.1). That is, a GP works in **function space**.

17.2.5 Numerical issues

In this section, we discuss computational and numerical issues which arise when implementing the above equations. For notational simplicity, we assume the prior mean is zero, $m(\mathbf{x}) = 0$.

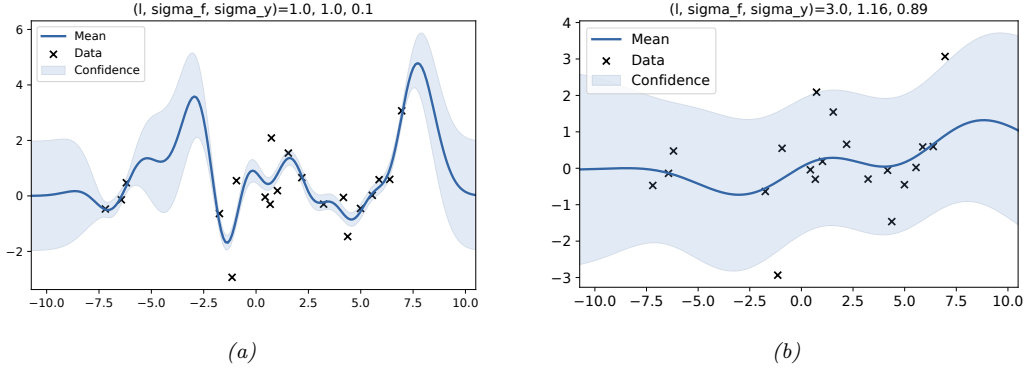


Figure 17.8: Some 1d GPs with SE kernels but different hyper-parameters fit to 20 noisy observations. The hyper-parameters $(\ell, \sigma_f, \sigma_y)$ are as follows: (a) $(1, 1.0, 0.1)$ (b) $(3.0, 1.16, 0.89)$. Adapted from Figure 2.5 of [RW06]. Generated by [gprDemoChangeHparams.ipynb](#).

The posterior predictive mean is given by $\mu_* = \mathbf{k}_*^\top \mathbf{K}_\sigma^{-1} \mathbf{y}$. For reasons of numerical stability, it is unwise to directly invert $\mathbf{K}_\sigma$. A more robust alternative is to compute a Cholesky decomposition, $\mathbf{K}_\sigma = \mathbf{L}\mathbf{L}^\top$, which takes $O(N^3)$ time. Then we compute $\boldsymbol{\alpha} = \mathbf{L}^\top \setminus (\mathbf{L} \setminus \mathbf{y})$, where we have used the backslash operator to represent backsubstitution (Section 7.7.1). Given this, we can compute the posterior mean for each test case in $O(N)$ time using

$$\mu_* = \mathbf{k}_*^\top \mathbf{K}_\sigma^{-1} \mathbf{y} = \mathbf{k}_*^\top \mathbf{L}^{-\top} (\mathbf{L}^{-1} \mathbf{y}) = \mathbf{k}_*^\top \boldsymbol{\alpha} \quad (17.47)$$

We can compute the variance in $O(N^2)$ time for each test case using

$$\sigma_*^2 = k_{**} - \mathbf{k}_*^\top \mathbf{L}^{-\top} \mathbf{L}^{-1} \mathbf{k}_* = k_{**} - \mathbf{v}^\top \mathbf{v} \quad (17.48)$$

where $\mathbf{v} = \mathbf{L} \setminus \mathbf{k}_*$.

Finally, the log marginal likelihood (needed for kernel learning, Section 17.2.6) can be computed using

$$\log p(\mathbf{y}|\mathbf{X}) = -\frac{1}{2} \mathbf{y}^\top \boldsymbol{\alpha} - \sum_{n=1}^N \log L_{nn} - \frac{N}{2} \log(2\pi) \quad (17.49)$$

17.2.6 Estimating the kernel

Most kernels have some free parameters, which can have a large effect on the predictions from the model. For example, suppose we are performing 1d regression using a GP with an RBF kernel of the form

$$\mathcal{K}(x_p, x_q) = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2} (x_p - x_q)^2\right) \quad (17.50)$$

Here ℓ is the horizontal scale over which the function changes, σ_f^2 controls the vertical scale of the function. We assume observation noise with variance σ_y^2 .

We sampled 20 observations from an MVN with a covariance given by $\Sigma = \mathcal{K}(x_i, x_j)$ for a grid of points $\{x_i\}$, and added observation noise of value σ_y . We then fit this data using a GP with the same kernel, but with a range of hyperparameters. Figure 17.8 illustrates the effects of changing these parameters. In Figure 17.8(a), we use $(\ell, \sigma_f, \sigma_y) = (1, 1, 0.1)$, and the result is a good fit. In Figure 17.8(b), we increase the length scale to $\ell = 3$; now the function looks overly smooth.

17.2.6.1 Empirical Bayes

To estimate the kernel parameters θ (sometimes called hyperparameters), we could use exhaustive search over a discrete grid of values, with validation loss as an objective, but this can be quite slow. (This is the approach used by nonprobabilistic methods, such as SVMs (Section 17.3) to tune kernels.) Here we consider an empirical Bayes approach (Section 4.6.5.3), which will allow us to use gradient-based optimization methods, which are much faster. In particular, we will maximize the marginal likelihood

$$p(\mathbf{y}|\mathbf{X}, \theta) = \int p(\mathbf{y}|\mathbf{f}, \mathbf{X})p(\mathbf{f}|\mathbf{X}, \theta)d\mathbf{f} \quad (17.51)$$

(The reason it is called the marginal likelihood, rather than just likelihood, is because we have marginalized out the latent Gaussian vector $\mathbf{f}$.)

For notational simplicity, we assume the mean function is 0. Since $p(\mathbf{f}|\mathbf{X}) = \mathcal{N}(\mathbf{f}|\mathbf{0}, \mathbf{K})$, and $p(\mathbf{y}|\mathbf{f}) = \prod_{n=1}^N \mathcal{N}(y_n|f_n, \sigma_y^2)$, the marginal likelihood is given by

$$\log p(\mathbf{y}|\mathbf{X}, \theta) = \log \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{K}_\sigma) = -\frac{1}{2}\mathbf{y}^\top \mathbf{K}_\sigma^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}_\sigma| - \frac{N}{2} \log(2\pi) \quad (17.52)$$

where the dependence of $\mathbf{K}_\sigma = \mathbf{K}_{X,X} + \sigma_y^2 \mathbf{I}_N$ on θ is implicit. The first term is a data fit term, the second term is a model complexity term, and the third term is just a constant. To understand the tradeoff between the first two terms, consider a SE kernel in 1D, as we vary the length scale ℓ and hold σ_y^2 fixed. For short length scales, the fit will be good, so $\mathbf{y}^\top \mathbf{K}_\sigma^{-1} \mathbf{y}$ will be small. However, the model complexity will be high: $\mathbf{K}$ will be almost diagonal, (as in Figure 13.22, top right), since most points will not be considered “near” any others, so the $\log |\mathbf{K}_\sigma|$ term will be large. For long length scales, the fit will be poor but the model complexity will be low: $\mathbf{K}$ will be almost all 1’s, (as in Figure 13.22, bottom right), so $\log |\mathbf{K}_\sigma|$ will be small.

We now discuss how to maximize the marginal likelihood. One can show that

$$\frac{\partial}{\partial \theta_j} \log p(\mathbf{y}|\mathbf{X}, \theta) = \frac{1}{2} \mathbf{y}^\top \mathbf{K}_\sigma^{-1} \frac{\partial \mathbf{K}_\sigma}{\partial \theta_j} \mathbf{K}_\sigma^{-1} \mathbf{y} - \frac{1}{2} \text{tr}(\mathbf{K}_\sigma^{-1} \frac{\partial \mathbf{K}_\sigma}{\partial \theta_j}) \quad (17.53)$$

$$= \frac{1}{2} \text{tr} \left((\boldsymbol{\alpha} \boldsymbol{\alpha}^\top - \mathbf{K}_\sigma^{-1}) \frac{\partial \mathbf{K}_\sigma}{\partial \theta_j} \right) \quad (17.54)$$

where $\boldsymbol{\alpha} = \mathbf{K}_\sigma^{-1} \mathbf{y}$. It takes $O(N^3)$ time to compute $\mathbf{K}_\sigma^{-1}$, and then $O(N^2)$ time per hyper-parameter to compute the gradient.

The form of $\frac{\partial \mathbf{K}_\sigma}{\partial \theta_j}$ depends on the form of the kernel, and which parameter we are taking derivatives with respect to. Often we have constraints on the hyper-parameters, such as $\sigma_y^2 \geq 0$. In this case, we can define $\theta = \log(\sigma_y^2)$, and then use the chain rule.

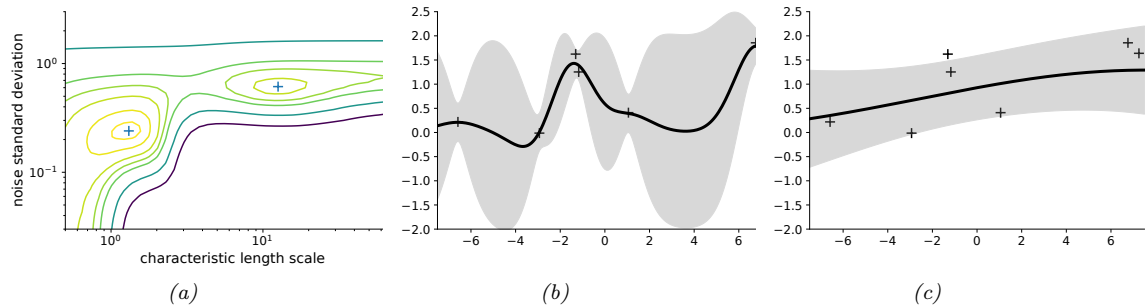


Figure 17.9: Illustration of local minima in the marginal likelihood surface. (a) We plot the log marginal likelihood vs kernel length scale ℓ and observation noise σ_y , for fixed signal level $\sigma_f = 1$, using the 7 data points shown in panels b and c. (b) The function corresponding to the lower left local minimum, $(\ell, \sigma_y) \approx (1, 0.2)$. This is quite “wiggly” and has low noise. (c) The function corresponding to the top right local minimum, $(\ell, \sigma_y) \approx (10, 0.8)$. This is quite smooth and has high noise. The data was generated using $(\ell, \sigma_f, \sigma_y) = (1, 1, 0.1)$. Adapted from Figure 5.5 of [RW06]. Generated by `gpr_demo_marglik.ipynb`.

Given an expression for the log marginal likelihood and its derivative, we can estimate the kernel parameters using any standard gradient-based optimizer. However, since the objective is not convex, local minima can be a problem, as we illustrate below, so we may need to use multiple restarts.

As an example, consider the RBF in Equation (17.50) with $\sigma_f^2 = 1$. In Figure 17.9(a), we plot $\log p(\mathbf{y}|\mathbf{X}, \ell, \sigma_y^2)$ (where $\mathbf{X}$ and $\mathbf{y}$ are the 7 data points shown in panels b and c) as we vary ℓ and σ_y^2 . The two local optima are indicated by +. The bottom left optimum corresponds to a low-noise, short-length scale solution (shown in panel b). The top right optimum corresponds to a high-noise, long-length scale solution (shown in panel c). With only 7 data points, there is not enough evidence to confidently decide which is more reasonable, although the more complex model (panel b) has a marginal likelihood that is about 60% higher than the simpler model (panel c). With more data, the more complex model would become even more preferred.

Figure 17.9 illustrates some other interesting (and typical) features. The region where $\sigma_y^2 \approx 1$ (top of panel a) corresponds to the case where the noise is very high; in this regime, the marginal likelihood is insensitive to the length scale (indicated by the horizontal contours), since all the data is explained as noise. The region where $\ell \approx 0.5$ (left hand side of panel a) corresponds to the case where the length scale is very short; in this regime, the marginal likelihood is insensitive to the noise level (indicated by the vertical contours), since the data is perfectly interpolated. Neither of these regions would be chosen by a good optimizer.

17.2.6.2 Bayesian inference

When we have a small number of datapoints (e.g., when using GPs for Bayesian optimization), using a point estimate of the kernel parameters can give poor results [Bul11; WF14]. In such cases, we may wish to approximate the posterior over the kernel parameters. Several methods can be used. For example, [MA10] shows how to use slice sampling, [Hen+15] shows how to use Hamiltonian Monte Carlo, and [BBV11] shows how to use sequential Monte Carlo.

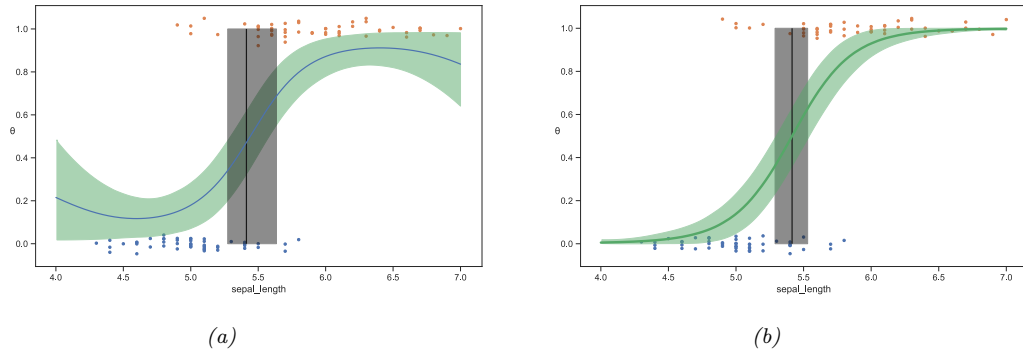


Figure 17.10: GP classifier for a binary classification problem on Iris flowers (Setosa vs Versicolor) using a single input feature (sepal length). The fat vertical line is the credible interval for the decision boundary. (a) SE kernel. (b) SE plus linear kernel. Adapted from Figures 7.11–7.12 of [Mar18]. Generated by `gp_classify_iris_1d_pymc3.ipynb`.

17.2.7 GPs for classification

So far, we have focused on GPs for regression using Gaussian likelihoods. In this case, the posterior is also a GP, and all computation can be performed analytically. However, if the likelihood is non-Gaussian, such as the Bernoulli likelihood for binary classification, we can no longer compute the posterior exactly.

There are various approximations we can make, some of which we discuss in the sequel to this book, [Mur23]. In this section, we use the Hamiltonian Monte Carlo method (Section 4.6.8.4), both for the latent Gaussian function $\mathbf{f}$ as well as the kernel hyperparameters $\boldsymbol{\theta}$. The basic idea is to specify the negative log joint

$$-\mathcal{E}(\mathbf{f}, \boldsymbol{\theta}) = \log p(\mathbf{f}, \boldsymbol{\theta} | \mathbf{X}, \mathbf{y}) = \log \mathcal{N}(\mathbf{f} | \mathbf{0}, \mathbf{K}(\mathbf{X}, \mathbf{X})) + \sum_{n=1}^N \log \text{Ber}(y_n | f_n(\mathbf{x}_n)) + \log p(\boldsymbol{\theta}) \quad (17.55)$$

We then use autograd to compute $\nabla_{\mathbf{f}} \mathcal{E}(\mathbf{f}, \boldsymbol{\theta})$ and $\nabla_{\boldsymbol{\theta}} \mathcal{E}(\mathbf{f}, \boldsymbol{\theta})$, and use these gradients as inputs to a Gaussian proposal distribution.

Let us consider a 1d example from [Mar18]. This is similar to the Bayesian logistic regression example from Figure 4.20, where the goal is to classify iris flowers as being Setosa or Versicolor, $y_n \in \{0, 1\}$, given information about the sepal length, x_n . We will use an SE kernel with length scale ℓ . We put a $\text{Ga}(2, 0.5)$ prior on ℓ .

Figure 17.10a shows the results using the SE kernel. This is similar to the results of linear logistic regression (see Figure 4.20), except that at the edges (away from the data), the probability curves towards 0.5. This is because the prior mean function is $m(x) = 0$, and $\sigma(0) = 0.5$. We can eliminate this artefact by using a more flexible kernel, which encodes the prior knowledge that we expect the output to be monotonically increasing or decreasing in the input. We can do this using a **linear kernel**,

$$\mathcal{K}(x, x') = (x - c)(x' - c) \quad (17.56)$$

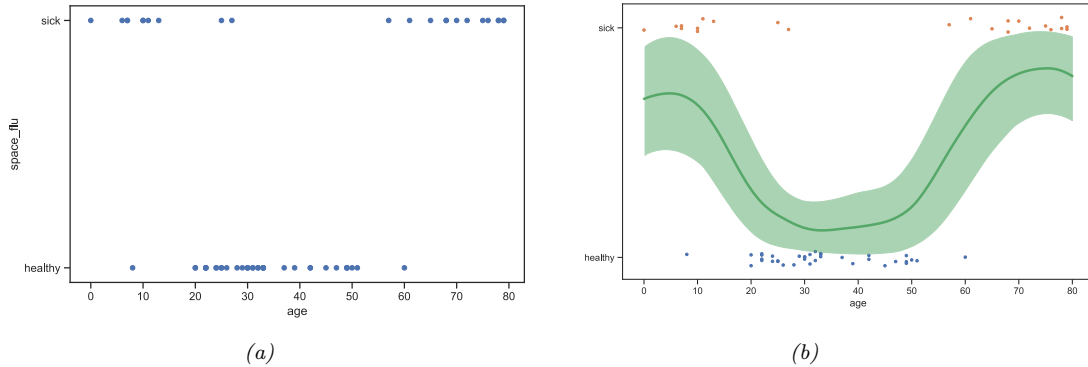


Figure 17.11: (a) Fictitious “space flu” binary classification problem. (b) Fit from a GP with SE kernel. Adapted from Figures 7.13–7.14 of [Mar18]. Generated by `gp_classify_spaceflu_1d_pymc3.ipynb`.

We can scale and add this to the SE kernel to get

$$\mathcal{K}(x, x') = \tau(x - c)(x' - c) + \exp \left[-\frac{(x - x')^2}{2\ell^2} \right] \quad (17.57)$$

The results are shown in Figure 17.10b, and look more reasonable.

One might wonder why we bothered to use a GP, when the results are no better than a simple linear logistic regression model. The reason is that the GP is much more flexible, and makes fewer a priori assumptions, beyond smoothness. For example, suppose the data looked like Figure 17.11a. In this case, a linear logistic regression model could not fit the data. We could in principle use a neural network, but it may not work well since we only have 60 data points. However, GPs are well designed to handle the small sample setting. In Figure 17.11b, we show the results of fitting a GP with an SE kernel to this data. The results look reasonable.

17.2.8 Connections with deep learning

It turns out that there are many interesting connections and similarities between GPs and deep neural networks. For example, one can show that a neural network with a single, **infinitely wide** layer of RBF units is equivalent to a GP with an RBF kernel. (This follows from the fact that the RBF kernel can be expressed as the inner product of an infinite number of features.) In fact, many kinds of DNNs (in the infinite limit) can be converted to an equivalent GP using a specific kind of kernel known as the **neural tangent kernel** [JGH18]. See the sequel to this book, [Mur23], for details.

17.2.9 Scaling GPs to large datasets

The main disadvantage of GPs (and other kernel methods, such as SVMs, which we discuss in Section 17.3) is that inverting the $N \times N$ kernel matrix takes $O(N^3)$ time, making the method too

slow for big datasets. Many different approximate schemes have been proposed to speedup GPs (see e.g., [Liu+18a] for a review). In this section, we briefly mention some of them. For more details, see the sequel to this book, [Mur23].

17.2.9.1 Sparse (inducing-point) approximations

A simple approach to speeding up GP inference is to use less data. A better approach is to try to “summarize” the N training points $\mathbf{X}$ into $M \ll N$ **inducing points** or **pseudo inputs** $\mathbf{Z}$. This lets us replace $p(\mathbf{f}|\mathbf{f}_X)$ with $p(\mathbf{f}|\mathbf{f}_Z)$, where $\mathbf{f}_X = \{f(\mathbf{x}) : \mathbf{x} \in \mathbf{Z}\}$ is the vector of observed function values at the training points, and $\mathbf{f}_Z = \{f(\mathbf{x}) : \mathbf{x} \in \mathbf{Z}\}$ is the vector of estimated function values at the inducing points. By optimizing $(\mathbf{Z}, \mathbf{f}_Z)$ we can learn to “compress” the training data $(\mathbf{X}, \mathbf{f}_X)$ into a “bottleneck” $(\mathbf{Z}, \mathbf{f}_Z)$, thus speeding up computation from $O(N^3)$ to $O(M^3)$. This is called a **sparse GP**. This whole process can be made rigorous using the framework of variational inference. For details, see the sequel to this book, [Mur23].

17.2.9.2 Exploiting parallelization and kernel matrix structure

It takes $O(N^3)$ time to compute the Cholesky decomposition of $\mathbf{K}_{X,X}$, which is needed to solve the linear system $\mathbf{K}_\sigma \boldsymbol{\alpha} = \mathbf{y}$ and to compute $|\mathbf{K}_{X,X}|$, where $\mathbf{K}_\sigma = \mathbf{K}_{X,X} + \sigma^2 \mathbf{I}_N$. An alternative to Cholesky decomposition is to use linear algebra methods, often called **Krylov subspace methods**, which are based just on **matrix vector multiplication** or **MVM**. These approaches are often much faster, since they can naturally exploit structure in the kernel matrix. Moreover, even if the kernel matrix does not have special structure, matrix multiplies are trivial to parallelize, and can thus be greatly accelerated by GPUs, unlike Cholesky based methods which are largely sequential. This is the basis of the popular **GPyTorch** package [Gar+18]. For more details, see the sequel to this book, [Mur23].

17.2.9.3 Random feature approximation

Although the power of kernels resides in the ability to avoid working with featurized representations of the inputs, such kernelized methods take $O(N^3)$ time, in order to invert the Gram matrix $\mathbf{K}$. This can make it difficult to use such methods on large scale data. Fortunately, we can approximate the feature map for many (shift invariant) kernels using a randomly chosen finite set of M basis functions, thus reducing the cost to $O(NM + M^3)$. We briefly discuss this idea below. For more details, see e.g., [Liu+20].

Random features for RBF kernel

We will focus on the case of the Gaussian RBF kernel. One can show that

$$\mathcal{K}(\mathbf{x}, \mathbf{x}') \approx \boldsymbol{\phi}(\mathbf{x})^\top \boldsymbol{\phi}(\mathbf{x}') \quad (17.58)$$

where the (real-valued) feature vector is given by

$$\boldsymbol{\phi}(\mathbf{x}) \triangleq \frac{1}{\sqrt{T}} [(\sin(\boldsymbol{\omega}_1^\top \mathbf{x}), \dots, \sin(\boldsymbol{\omega}_T^\top \mathbf{x}), \cos(\boldsymbol{\omega}_1^\top \mathbf{x}), \dots, \cos(\boldsymbol{\omega}_T^\top \mathbf{x}))] \quad (17.59)$$

$$= \frac{1}{\sqrt{T}} [\sin(\boldsymbol{\Omega} \mathbf{x}), \cos(\boldsymbol{\Omega} \mathbf{x})] \quad (17.60)$$

where $T = M/2$, and $\mathbf{\Omega} \in \mathbb{R}^{T \times D}$ is a random Gaussian matrix, where the entries are sampled iid from $\mathcal{N}(0, 1/\sigma^2)$, where σ is the kernel bandwidth. The bias of the approximation decreases as we increase M . In practice, we use a finite M , and compute a single sample Monte Carlo approximation to the expectation by drawing a single random matrix. The features in Equation (17.60) are called **random Fourier features (RFF)** [RR08] or “weighted sums of random kitchen sinks” [RR09].

We can also use positive random features, rather than trigonometric random features, which can be preferable in some applications, such as models which use attention (see Section 15.6.4). In particular, we can use

$$\phi(\mathbf{x}) \triangleq e^{-\|\mathbf{x}\|^2/2} \frac{1}{\sqrt{M}} [(\exp(\boldsymbol{\omega}_1^\top \mathbf{x}), \dots, (\exp(\boldsymbol{\omega}_M^\top \mathbf{x})) \quad (17.61)$$

where $\boldsymbol{\omega}_m$ are sampled as before. For details, see [Cho+20b].

Regardless of whether we use trigonometric or positive features, we can obtain a lower variance estimate by ensuring that the rows of $\mathbf{\Omega}$ are random but orthogonal; these are called **orthogonal random features**. Such sampling can be conducted efficiently via Gram-Schmidt orthogonalization of the unstructured Gaussian matrices [Yu+16], or several approximations that are even faster (see [CRW17; Cho+19]).

Fastfood approximation

Unfortunately, storing the random matrix $\mathbf{\Omega}$ takes $O(DM)$ space, and computing $\mathbf{\Omega}\mathbf{x}$ takes $O(DM)$ time, where D is the input dimensionality, and M is the number of random features. This can be prohibitive if $M \gg D$, which it may need to be in order to get any benefits over using the original set of features. Fortunately, we can use the **fast Hadamard transform** to reduce the memory from $O(MD)$ to $O(M)$, and reduce the time from $O(MD)$ to $O(M \log D)$. This approach has been called **fastfood** [LSS13], a reference to the original term “kitchen sinks”.

Extreme learning machines

We can use the random features approximation to the kernel to convert a GP into a linear model of the form

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}\phi(\mathbf{x}) = \mathbf{W}h(\mathbf{\Omega}\mathbf{x}) \quad (17.62)$$

where $h(a) = \sqrt{1/M}[\sin(a), \cos(a)]$ for RBF kernels. This is equivalent to a one-layer MLP with random (and fixed) input-to-hidden weights. When $M > N$, this corresponds to an over-parameterized model, which can perfectly interpolate the training data.

In [Cur+17], they apply this method to fit a logistic regression model of the form $f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{W}^\top h(\hat{\mathbf{\Omega}}\mathbf{x}) + \mathbf{b}$ using SGD; they call the resulting method “**McKernel**”. We can also optimize $\mathbf{\Omega}$ as well as $\mathbf{W}$, as discussed in [Alb+17], although now the problem is no longer convex.

Alternatively, we can use $M < N$, but stack many such random nonlinear layers together, and just optimize the output weights. This has been called an **extreme learning machine** or **ELM** (see e.g., [Hua14]), although this work is controversial.¹

1. The controversy has arisen because the inventor Guang-Bin Huang has been accused of not citing related prior work, such as the equivalent approach based on random feature approximations to kernels. For details, see https://en.wikipedia.org/wiki/Extreme_learning_machine#Controversy.

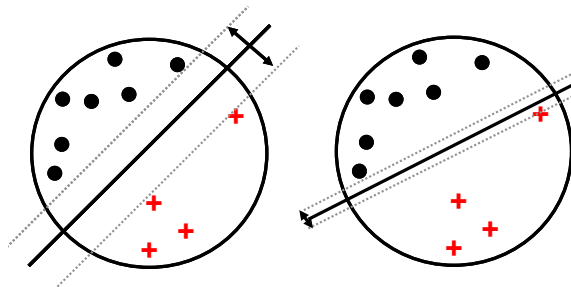


Figure 17.12: Illustration of the large margin principle. Left: a separating hyper-plane with large margin. Right: a separating hyper-plane with small margin.

17.3 Support vector machines (SVMs)

In this section, we discuss a form of (non-probabilistic) predictors for classification and regression problems which have the form

$$f(\mathbf{x}) = \sum_{i=1}^N \alpha_i \mathcal{K}(\mathbf{x}, \mathbf{x}_i) \quad (17.63)$$

By adding suitable constraints, we can ensure that many of the α_i coefficients are 0, so that predictions at test time only depend on a subset of the training points, known as “**support vectors**”. Hence the resulting model is called a **support vector machine** or **SVM**. We give a brief summary below. More details, can be found in e.g., [VGS97; SS01].

17.3.1 Large margin classifiers

Consider a binary classifier of the form $h(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$, where the decision boundary is given by the following linear function:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 \quad (17.64)$$

(In the SVM literature, it is common to assume the class labels are -1 and $+1$, rather than 0 and 1 . To avoid confusion, we denote such target labels by $\tilde{y}$ rather than y .) There may be many lines that separate the data. However, intuitively we would like to pick the one that has maximum **margin**, which is the distance of the closest point to the decision boundary, since this will give us the most robust solution. This idea is illustrated in Figure 17.12: the solution on the left has larger margin than the one on the right, so it will be less sensitive to perturbations of the data.

How can we compute such a **large margin classifier**? First we need to derive an expression for the distance of a point to the decision boundary. Referring to Figure 17.13(a), we see that

$$\mathbf{x} = \mathbf{x}_\perp + r \frac{\mathbf{w}}{\|\mathbf{w}\|} \quad (17.65)$$

where r is the distance of $\mathbf{x}$ from the decision boundary whose normal vector is $\mathbf{w}$, and $\mathbf{x}_\perp$ is the orthogonal projection of $\mathbf{x}$ onto this boundary.

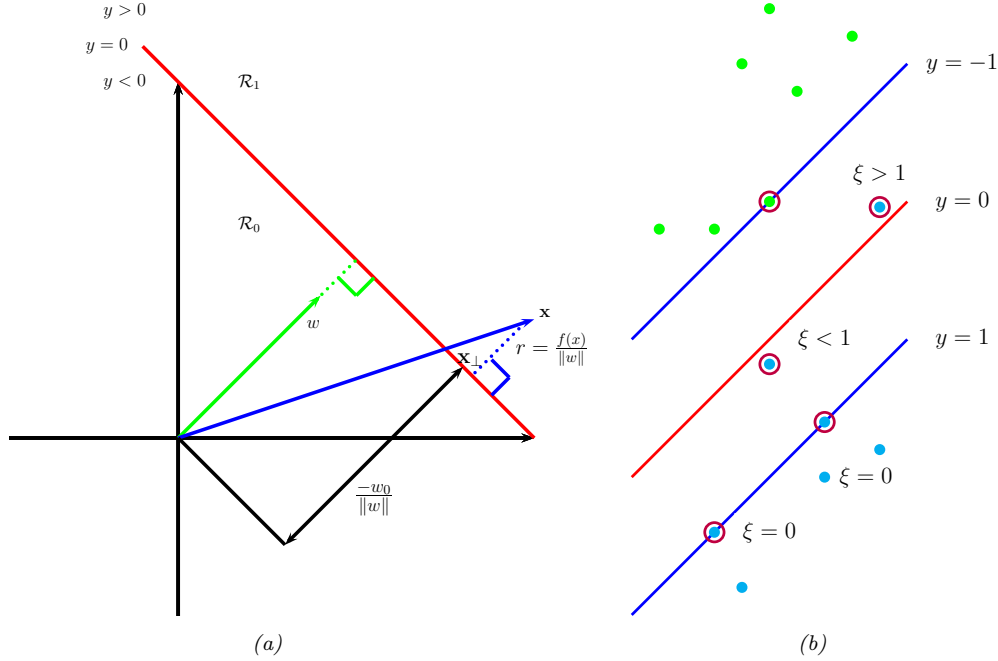


Figure 17.13: (a) Illustration of the geometry of a linear decision boundary in 2d. A point $\mathbf{x}$ is classified as belonging in decision region $\mathcal{R}_1$ if $f(\mathbf{x}) > 0$, otherwise it belongs in decision region $\mathcal{R}_0$; $\mathbf{w}$ is a vector which is perpendicular to the decision boundary. The term w_0 controls the distance of the decision boundary from the origin. $\mathbf{x}_\perp$ is the orthogonal projection of $\mathbf{x}$ onto the boundary. The signed distance of $\mathbf{x}$ from the boundary is given by $f(\mathbf{x})/\|\mathbf{w}\|$. Adapted from Figure 4.1 of [Bis06]. (b) Points with circles around them are support vectors, and have dual variables $\alpha_n > 0$. In the soft margin case, we associate a slack variable ξ_n with each example. If $0 < \xi_n < 1$, the point is inside the margin, but on the correct side of the decision boundary. If $\xi_n > 1$, the point is on the wrong side of the boundary. Adapted from Figure 7.3 of [Bis06].

We would like to maximize r , so we need to express it as a function of $\mathbf{w}$. First, note that

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 = (\mathbf{w}^\top \mathbf{x}_\perp + w_0) + r \frac{\mathbf{w}^\top \mathbf{w}}{\|\mathbf{w}\|} = (\mathbf{w}^\top \mathbf{x}_\perp + w_0) + r \|\mathbf{w}\| \quad (17.66)$$

Since $0 = f(\mathbf{x}_\perp) = \mathbf{w}^\top \mathbf{x}_\perp + w_0$, we have $f(\mathbf{x}) = r \|\mathbf{w}\|$ and hence $r = \frac{f(\mathbf{x})}{\|\mathbf{w}\|}$.

Since we want to ensure each point is on the correct side of the boundary, we also require $f(\mathbf{x}_n) \tilde{y}_n > 0$. We want to maximize the distance of the closest point, so our final objective becomes

$$\max_{\mathbf{w}, w_0} \frac{1}{\|\mathbf{w}\|} \min_{n=1}^N [\tilde{y}_n (\mathbf{w}^\top \mathbf{x}_n + w_0)] \quad (17.67)$$

Note that by rescaling the parameters using $\mathbf{w} \rightarrow k\mathbf{w}$ and $w_0 \rightarrow kw_0$, we do not change the distance of any point to the boundary, since the k factor cancels out when we divide by $\|\mathbf{w}\|$. Therefore let us define the scale factor such that $\tilde{y}_n f_n = 1$ for the point that is closest to the decision boundary.

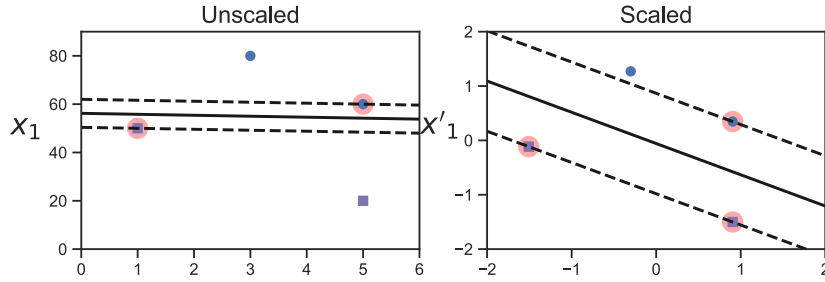


Figure 17.14: Illustration of the benefits of scaling the input features before computing a max margin classifier. Adapted from Figure 5.2 of [Gér19]. Generated by `svm_classifier_feature_scaling.ipynb`.

Hence we require $\tilde{y}_n f_n \geq 1$ for all n . Finally, note that maximizing $1/\|\mathbf{w}\|$ is equivalent to minimizing $\|\mathbf{w}\|^2$. Thus we get the new objective

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad \tilde{y}_n (\mathbf{w}^\top \mathbf{x}_n + w_0) \geq 1, n = 1 : N \quad (17.68)$$

(The factor of $\frac{1}{2}$ is added for convenience and doesn't affect the optimal parameters.) The constraint says that we want all points to be on the correct side of the decision boundary with a margin of at least 1.

Note that it is important to scale the input variables before using an SVM, otherwise the margin measures distance of a point to the boundary using all input dimensions equally. See Figure 17.14 for an illustration.

17.3.2 The dual problem

The objective in Equation (17.68) is a standard quadratic programming problem (Section 8.5.4), since we have a quadratic objective subject to linear constraints. This has $N + D + 1$ variables subject to N constraints, and is known as a **primal problem**.

In convex optimization, for every primal problem we can derive a **dual problem**. Let $\boldsymbol{\alpha} \in \mathbb{R}^N$ be the dual variables, corresponding to Lagrange multipliers that enforce the N inequality constraints. The generalized Lagrangian is given below (see Section 8.5.2 for relevant background information on constrained optimization):

$$\mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) = \frac{1}{2} \mathbf{w}^\top \mathbf{w} - \sum_{n=1}^N \alpha_n (\tilde{y}_n (\mathbf{w}^\top \mathbf{x}_n + w_0) - 1) \quad (17.69)$$

To optimize this, we must find a stationary point that satisfies

$$(\hat{\mathbf{w}}, \hat{w}_0, \hat{\boldsymbol{\alpha}}) = \min_{\mathbf{w}, w_0} \max_{\boldsymbol{\alpha}} \mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) \quad (17.70)$$

We can do this by computing the partial derivatives wrt $\mathbf{w}$ and w_0 and setting to zero. We have

$$\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) = \mathbf{w} - \sum_{n=1}^N \alpha_n \tilde{y}_n \mathbf{x}_n \quad (17.71)$$

$$\frac{\partial}{\partial w_0} \mathcal{L}(\mathbf{w}, w_0, \boldsymbol{\alpha}) = - \sum_{n=1}^N \alpha_n \tilde{y}_n \quad (17.72)$$

and hence

$$\hat{\mathbf{w}} = \sum_{n=1}^N \hat{\alpha}_n \tilde{y}_n \mathbf{x}_n \quad (17.73)$$

$$0 = \sum_{n=1}^N \hat{\alpha}_n \tilde{y}_n \quad (17.74)$$

Plugging these into the Lagrangian yields the following

$$\mathcal{L}(\hat{\mathbf{w}}, \hat{w}_0, \boldsymbol{\alpha}) = \frac{1}{2} \hat{\mathbf{w}}^\top \hat{\mathbf{w}} - \sum_{n=1}^N \alpha_n \tilde{y}_n \hat{\mathbf{w}}^\top \mathbf{x}_n - \sum_{n=1}^N \alpha_n \tilde{y}_n w_0 + \sum_{n=1}^N \alpha_n \quad (17.75)$$

$$= \frac{1}{2} \hat{\mathbf{w}}^\top \hat{\mathbf{w}} - \hat{\mathbf{w}}^\top \hat{\mathbf{w}} - 0 + \sum_{n=1}^N \alpha_n \quad (17.76)$$

$$= -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \mathbf{x}_i^\top \mathbf{x}_j + \sum_{n=1}^N \alpha_n \quad (17.77)$$

This is called the **dual form** of the objective. We want to maximize this wrt $\boldsymbol{\alpha}$ subject to the constraints that $\sum_{n=1}^N \alpha_n \tilde{y}_n = 0$ and $0 \leq \alpha_n$ for $n = 1 : N$.

The above objective is a quadratic problem in N variables. Standard QP solvers take $O(N^3)$ time. However, specialized algorithms, which avoid the use of generic QP solvers, have been developed for this problem, such as the **sequential minimal optimization** or **SMO** algorithm [Pla98], which takes $O(N)$ to $O(N^2)$ time.

Since this is a convex objective, the solution must satisfy the KKT conditions (Section 8.5.2), which tell us that the following properties hold:

$$\alpha_n \geq 0 \quad (17.78)$$

$$\tilde{y}_n f(\mathbf{x}_n) - 1 \geq 0 \quad (17.79)$$

$$\alpha_n (\tilde{y}_n f(\mathbf{x}_n) - 1) = 0 \quad (17.80)$$

Hence either $\alpha_n = 0$ (in which case example n is ignored when computing $\hat{\mathbf{w}}$) or the constraint $\tilde{y}_n (\hat{\mathbf{w}}^\top \mathbf{x}_n + \hat{w}_0) = 1$ is active. This latter condition means that example n lies on the decision boundary; these points are known as the **support vectors**, as shown in Figure 17.13(b). We denote the set of support vectors by $\mathcal{S}$.

To perform prediction, we use

$$f(\mathbf{x}; \hat{\mathbf{w}}, \hat{w}_0) = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \mathbf{x}_n^\top \mathbf{x} + \hat{w}_0 \quad (17.81)$$

To solve for $\hat{w}_0$ we can use the fact that for any support vector, we have $\tilde{y}_n f(\mathbf{x}; \hat{\mathbf{w}}, \hat{w}_0) = 1$. Multiplying both sides by $\tilde{y}_n$, and exploiting the fact that $\tilde{y}_n^2 = 1$, we get $\hat{w}_0 = \tilde{y}_n - \hat{\mathbf{w}}^\top \mathbf{x}_n$. In practice we get better results by averaging over all the support vectors to get

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} (\tilde{y}_n - \hat{\mathbf{w}}^\top \mathbf{x}_n) = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} (\tilde{y}_n - \sum_{m \in \mathcal{S}} \alpha_m \tilde{y}_m \mathbf{x}_m^\top \mathbf{x}_n) \quad (17.82)$$

17.3.3 Soft margin classifiers

If the data is not linearly separable, there will be no feasible solution in which $\tilde{y}_n f_n \geq 1$ for all n . We therefore introduce **slack variables** $\xi_n \geq 0$ and replace the hard constraints that $\tilde{y}_n f_n \geq 0$ with the **soft margin constraints** that $\tilde{y}_n f_n \geq 1 - \xi_n$. The new objective becomes

$$\min_{\mathbf{w}, w_0, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N \xi_n \quad \text{s.t.} \quad \xi_n \geq 0, \quad \tilde{y}_n (\mathbf{x}_n^\top \mathbf{w} + w_0) \geq 1 - \xi_n \quad (17.83)$$

where $C \geq 0$ is a hyper parameter controlling how many points we allow to violate the margin constraint. (If $C = \infty$, we recover the unregularized, hard-margin classifier.)

The corresponding Lagrangian for the soft margin classifier becomes

$$\mathcal{L}(\mathbf{w}, w_0, \alpha, \xi, \mu) = \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N \alpha_n (\tilde{y}_n (\mathbf{w}^\top \mathbf{x}_n + w_0) - 1 + \xi_n) - \sum_{n=1}^N \mu_n \xi_n \quad (17.84)$$

where $\alpha_n \geq 0$ and $\mu_n \geq 0$ are the Lagrange multipliers. Optimizing out $\mathbf{w}$, w_0 and ξ gives the dual form

$$\mathcal{L}(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \mathbf{x}_i^\top \mathbf{x}_j \quad (17.85)$$

This is identical to the hard margin case; however, the constraints are different. In particular, the KKT conditions imply

$$0 \leq \alpha_n \leq C \quad (17.86)$$

$$\sum_{n=1}^N \alpha_n \tilde{y}_n = 0 \quad (17.87)$$

If $\alpha_n = 0$, the point is ignored. If $0 < \alpha_n < C$ then $\xi_n = 0$, so the point lies on the margin. If $\alpha_n = C$, the point can lie inside the margin, and can either be correctly classified if $\xi_n \leq 1$, or misclassified if $\xi_n > 1$. See Figure 17.13(b) for an illustration. Hence $\sum_n \xi_n$ is an upper bound on the number of misclassified points.

As before, the bias term can be computed using

$$\hat{w}_0 = \frac{1}{|\mathcal{M}|} \sum_{n \in \mathcal{M}} (\tilde{y}_n - \sum_{m \in \mathcal{S}} \alpha_m \tilde{y}_m \mathbf{x}_m^\top \mathbf{x}_n) \quad (17.88)$$

where $\mathcal{M}$ is the set of points having $0 < \alpha_n < C$.

There is an alternative formulation of the soft margin SVM known as the ν -SVM classifier [Sch+00]. This involves maximizing

$$\mathcal{L}(\boldsymbol{\alpha}) = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j \tilde{y}_i \tilde{y}_j \mathbf{x}_i^\top \mathbf{x}_j \quad (17.89)$$

subject to the constraints that

$$0 \leq \alpha_n \leq 1/N \quad (17.90)$$

$$\sum_{n=1}^N \alpha_n \tilde{y}_n = 0 \quad (17.91)$$

$$\sum_{n=1}^M \alpha_n \geq \nu \quad (17.92)$$

This has the advantage that the parameter ν , which replaces C , can be interpreted as an upper bound on the fraction of **margin errors** (points for which $\xi_n > 0$), as well as a lower bound on the number of support vectors.

17.3.4 The kernel trick

So far we have converted the large margin binary classification problem into a dual problem in N unknowns ($\boldsymbol{\alpha}$) which (in general) takes $O(N^3)$ time to solve, which can be slow. However, the principal benefit of the dual problem is that we can replace all inner product operations $\mathbf{x}^\top \mathbf{x}'$ with a call to a positive definite (Mercer) kernel function, $\mathcal{K}(\mathbf{x}, \mathbf{x}')$. This is called the **kernel trick**.

In particular, we can rewrite the prediction function in Equation (17.81) as follows:

$$f(\mathbf{x}) = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \mathbf{x}_n^\top \mathbf{x} + \hat{w}_0 = \sum_{n \in \mathcal{S}} \alpha_n \tilde{y}_n \mathcal{K}(\mathbf{x}_n, \mathbf{x}) + \hat{w}_0 \quad (17.93)$$

We also need to kernelize the bias term. This can be done by kernelizing Equation (17.82) as follows:

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(\tilde{y}_i - \left(\sum_{j \in \mathcal{S}} \hat{\alpha}_j \tilde{y}_j \mathbf{x}_j \right)^\top \mathbf{x}_i \right) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(\tilde{y}_i - \sum_{j \in \mathcal{S}} \hat{\alpha}_j \tilde{y}_j \mathcal{K}(\mathbf{x}_j, \mathbf{x}_i) \right) \quad (17.94)$$

The kernel trick allows us to avoid having to deal with an explicit feature representation of our data, and allows us to easily apply classifiers to structured objects, such as strings and graphs.

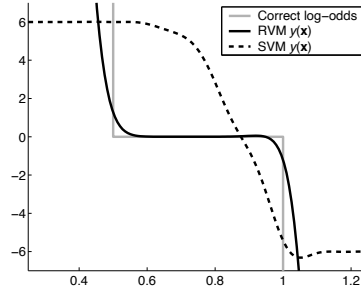


Figure 17.15: Log-odds vs x for 3 different methods. Adapted from Figure 10 of [Tip01]. Used with kind permission of Mike Tipping.

17.3.5 Converting SVM outputs into probabilities

An SVM classifier produces a hard-labeling, $\hat{y}(\mathbf{x}) = \text{sign}(f(\mathbf{x}))$. However, we often want a measure of confidence in our prediction. One heuristic approach is to interpret $f(\mathbf{x})$ as the log-odds ratio, $\log \frac{p(y=1|\mathbf{x})}{p(y=0|\mathbf{x})}$. We can then convert the output of an SVM to a probability using

$$p(y=1|\mathbf{x}, \boldsymbol{\theta}) = \sigma(a f(\mathbf{x}) + b) \quad (17.95)$$

where a, b can be estimated by maximum likelihood on a separate validation set. (Using the training set to estimate a and b leads to severe overfitting.) This technique was first proposed in [Pla00], and is known as **Platt scaling**.

However, the resulting probabilities are not particularly well calibrated, since there is nothing in the SVM training procedure that justifies interpreting $f(\mathbf{x})$ as a log-odds ratio. To illustrate this, consider an example from [Tip01]. Suppose we have 1d data where $p(x|y=0) = \text{Unif}(0,1)$ and $p(x|y=1) = \text{Unif}(0.5, 1.5)$. Since the class-conditional distributions overlap in the $[0.5, 1]$ range, the log-odds of class 1 over class 0 should be zero in this region, and infinite outside this region. We sampled 1000 points from the model, and then fit a probabilistic kernel classifier (an RVM, described in Section 17.4.1) and an SVM with a Gaussian kernel of width 0.1. Both models can perfectly capture the decision boundary, and achieve a generalization error of 25%, which is Bayes optimal in this problem. The probabilistic output from the RVM is a good approximation to the true log-odds, but this is not the case for the SVM, as shown in Figure 17.15.

17.3.6 Connection with logistic regression

We have seen that data points that are on the correct side of the decision boundary have $\xi_n = 0$; for the others, we have $\xi_n = 1 - \tilde{y}_n f(\mathbf{x}_n)$. Therefore we can rewrite the objective in Equation (17.83) as follows:

$$\mathcal{L}(\mathbf{w}) = \sum_{n=1}^N \ell_{\text{hinge}}(\tilde{y}_n, f(\mathbf{x}_n)) + \lambda \|\mathbf{w}\|^2 \quad (17.96)$$

where $\lambda = (2C)^{-1}$ and $\ell_{\text{hinge}}(y, \eta)$ is the **hinge loss** function defined by

$$\ell_{\text{hinge}}(\tilde{y}, \eta) = \max(0, 1 - \tilde{y}\eta) \quad (17.97)$$

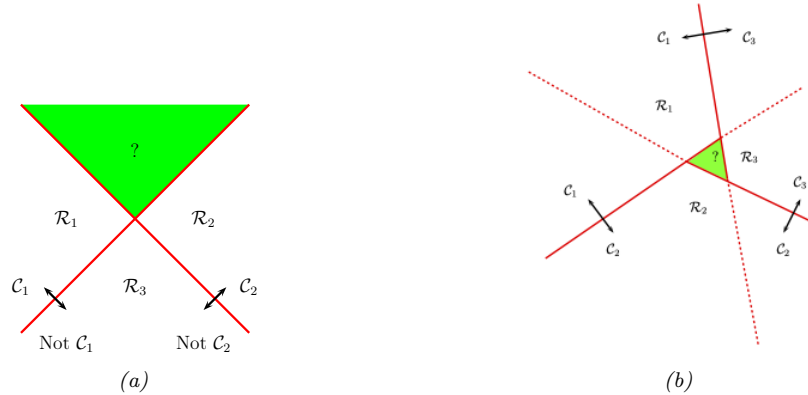


Figure 17.16: (a) The one-versus-rest approach. The green region is predicted to be both class 1 and class 2. (b) The one-versus-one approach. The label of the green region is ambiguous. Adapted from Figure 4.2 of [Bis06].

As we see from Figure 4.2, this is a convex, piecewise differentiable upper bound to the 0-1 loss, that has the shape of a partially open door hinge.

By contrast, (penalized) logistic regression optimizes

$$\mathcal{L}(\mathbf{w}) = \sum_{n=1}^N \ell_u(\tilde{y}_n, f(\mathbf{x}_n)) + \lambda \|\mathbf{w}\|^2 \quad (17.98)$$

where the **log loss** is given by

$$\ell_u(\tilde{y}, \eta) = -\log p(y|\eta) = \log(1 + e^{-\tilde{y}\eta}) \quad (17.99)$$

This is also plotted in Figure 4.2. We see that it is similar to the hinge loss, but with two important differences. First the hinge loss is piecewise linear, so we cannot use regular gradient methods to optimize it. (We can, however, compute the subgradient at $\tilde{y}\eta = 1$.) Second, the hinge loss has a region where it is strictly 0; this results in sparse estimates.

We see that both functions are convex upper bounds on the 0-1 loss, which is given by

$$\ell_{01}(\tilde{y}, \hat{y}) = \mathbb{I}(\tilde{y} \neq \hat{y}) = \mathbb{I}(\tilde{y} \hat{y} < 0) \quad (17.100)$$

These upper bounds are easier to optimize and can be viewed as surrogates for the 0-1 loss. See Section 4.3.2 for details.

17.3.7 Multi-class classification with SVMs

SVMs are inherently a binary classifier. One way to convert them to a multi-class classification model is to train C binary classifiers, where the data from class c is treated as positive, and the data from all the other classes is treated as negative. We then use the rule $\hat{y}(\mathbf{x}) = \arg \max_c f_c(\mathbf{x})$ to

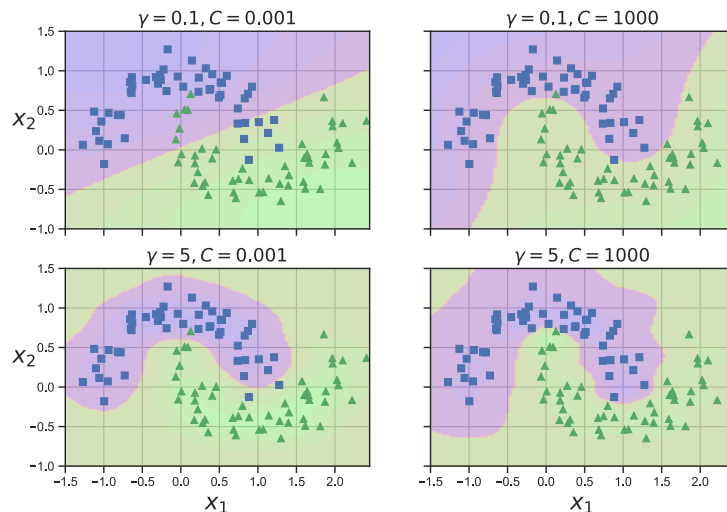


Figure 17.17: SVM classifier with RBF kernel with precision γ and regularizer C applied to two moons data. Adapted from Figure 5.9 of [Gér19]. Generated by `svm_classifier_2d.ipynb`.

predict the final label, where $f_c(\mathbf{x}) = \log \frac{p(c=1|\mathbf{x})}{p(c=0|\mathbf{x})}$ is the score given by classifier c . This is known as the **one-versus-the-rest** approach (also called **one-vs-all**).

Unfortunately, this approach has several problems. First, it can result in regions of input space which are ambiguously labeled. For example, the green region at the top of Figure 17.16(a) is predicted to be both class 2 and class 1. A second problem is that the magnitude of the f_c 's scores are not calibrated with each other, so it is hard to compare them. Finally, each binary subproblem is likely to suffer from the class imbalance problem (Section 10.3.8.2). For example, suppose we have 10 equally represented classes. When training f_1 , we will have 10% positive examples and 90% negative examples, which can hurt performance.

Another approach is to use the **one-versus-one** or OVO approach, also called **all pairs**, in which we train $C(C-1)/2$ classifiers to discriminate all pairs $f_{c,c'}$. We then classify a point into the class which has the highest number of votes. However, this can also result in ambiguities, as shown in Figure 17.16(b). Also, this requires fitting $O(C^2)$ models.

17.3.8 How to choose the regularizer C

SVMs require that you specify the kernel function and the parameter C . Typically C is chosen by cross-validation. Note, however, that C interacts quite strongly with the kernel parameters. For example, suppose we are using an RBF kernel with precision $\gamma = \frac{1}{2\sigma^2}$. If γ is large, corresponding to narrow kernels, we may need heavy regularization, and hence small C . If γ is small, a larger value of C should be used. So we see that γ and C are tightly coupled, as illustrated in Figure 17.17.

The authors of `libsvm` [HCL03] recommend using CV over a 2d grid with values $C \in \{2^{-5}, 2^{-3}, \dots, 2^{15}\}$

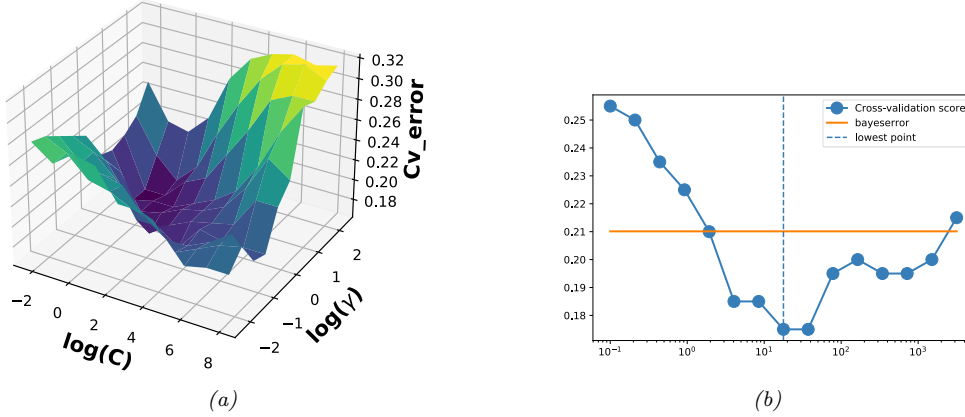


Figure 17.18: (a) A cross validation estimate of the 0-1 error for an SVM classifier with RBF kernel with different precisions $\gamma = 1/(2\sigma^2)$ and different regularizer $\lambda = 1/C$, applied to a synthetic data set drawn from a mixture of 2 Gaussians. (b) A slice through this surface for $\gamma = 5$. The red dotted line is the Bayes optimal error, computed using Bayes rule applied to the model used to generate the data. Adapted from Figure 12.6 of [HTF09]. Generated by [svmCgammaDemo.ipynb](#).

and $\gamma \in \{2^{-15}, 2^{-13}, \dots, 2^3\}$. See Figure 17.18 which shows the CV estimate of the 0-1 risk as a function of C and γ .

To choose C efficiently, one can develop a path following algorithm in the spirit of lars (Section 11.4.4). The basic idea is to start with C small, so that the margin is wide, and hence all points are inside of it and have $\alpha_i = 1$. By slowly increasing C , a small set of points will move from inside the margin to outside, and their α_i values will change from 1 to 0, as they cease to be support vectors. When C is maximal, the margin becomes empty, and no support vectors remain. See [Has+04] for the details.

17.3.9 Kernel ridge regression

Recall the equation for ridge regression from Equation (11.55):

$$\hat{\mathbf{w}}_{\text{map}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_D)^{-1} \mathbf{X}^T \mathbf{y} = \left(\sum_n \mathbf{x}_n \mathbf{x}_n^T + \lambda \mathbf{I}_D \right)^{-1} \left(\sum_n \tilde{y}_n \mathbf{x}_n \right) \quad (17.101)$$

Using the matrix inversion lemma (Section 7.3.3), we can rewrite the ridge estimate as follows

$$\mathbf{w} = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}_N)^{-1} \mathbf{y} = \sum_n \mathbf{x}_n \left(\left(\sum_n \mathbf{x}_n^T \mathbf{x}_n + \lambda \mathbf{I}_N \right)^{-1} \mathbf{y} \right)_n \quad (17.102)$$

Let us define the following **dual variables**:

$$\boldsymbol{\alpha} \triangleq (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}_N)^{-1} \mathbf{y} = \left(\sum_n \mathbf{x}_n^T \mathbf{x}_n + \lambda \mathbf{I}_N \right)^{-1} \mathbf{y} \quad (17.103)$$

Then we can rewrite the **primal variables** as follows

$$\mathbf{w} = \mathbf{X}^\top \boldsymbol{\alpha} = \sum_{n=1}^N \alpha_n \mathbf{x}_n \quad (17.104)$$

This tells us that the solution vector is just a linear sum of the N training vectors. When we plug this in at test time to compute the predictive mean, we get

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{w}^\top \mathbf{x} = \sum_{n=1}^N \alpha_n \mathbf{x}_n^\top \mathbf{x} \quad (17.105)$$

We can then use the kernel trick to rewrite this as

$$f(\mathbf{x}; \mathbf{w}) = \sum_{n=1}^N \alpha_n \mathcal{K}(\mathbf{x}_n, \mathbf{x}) \quad (17.106)$$

where

$$\boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{y} \quad (17.107)$$

In other words,

$$f(\mathbf{x}; \mathbf{w}) = \mathbf{k}^\top (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{y} \quad (17.108)$$

where $\mathbf{k} = [\mathcal{K}(\mathbf{x}, \mathbf{x}_1), \dots, \mathcal{K}(\mathbf{x}, \mathbf{x}_N)]$. This is called **kernel ridge regression**.

The trouble with the above approach is that the solution vector $\boldsymbol{\alpha}$ is not sparse, so predictions at test time will take $O(N)$ time. We discuss a solution to this in Section 17.3.10.

17.3.10 SVMs for regression

Consider the following ℓ_2 -regularized ERM problem:

$$J(\mathbf{w}, \lambda) = \lambda \|\mathbf{w}\|^2 + \sum_{n=1}^N \ell(\tilde{y}_n, \hat{y}_n) \quad (17.109)$$

where $\hat{y}_n = \mathbf{w}^\top \mathbf{x}_n + w_0$. If we use the quadratic loss, $\ell(y, \hat{y}) = (y - \hat{y})^2$, where $y, \hat{y} \in \mathbb{R}$, we recover ridge regression (Section 11.3). If we then apply the kernel trick, we recover kernel ridge regression (Section 17.3.9).

The problem with kernel ridge regression is that the solution depends on all N training points, which makes it computationally intractable. However, by changing the loss function, we can make the optimal set of basis function coefficients, $\boldsymbol{\alpha}^*$, be sparse, as we show below.

In particular, consider the following variant of the Huber loss function (Section 5.1.5.3) called the **epsilon insensitive loss function**:

$$L_\epsilon(y, \hat{y}) \triangleq \begin{cases} 0 & \text{if } |y - \hat{y}| < \epsilon \\ |y - \hat{y}| - \epsilon & \text{otherwise} \end{cases} \quad (17.110)$$

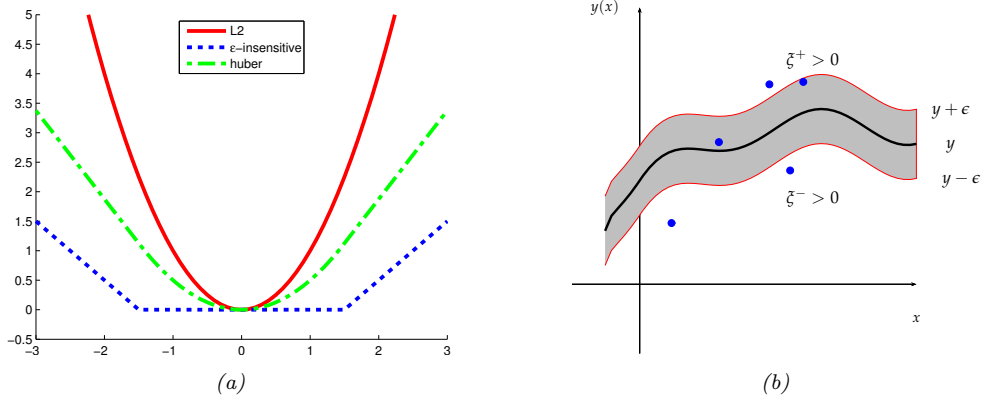


Figure 17.19: (a) Illustration of ℓ_2 , Huber and ϵ -insensitive loss functions, where $\epsilon = 1.5$. Generated by [huberLossPlot.ipynb](#). (b) Illustration of the ϵ -tube used in SVM regression. Points above the tube have $\xi_i^+ > 0$ and $\xi_i^- = 0$. Points below the tube have $\xi_i^+ = 0$ and $\xi_i^- > 0$. Points inside the tube have $\xi_i^+ = \xi_i^- = 0$. Adapted from Figure 7.7 of [Bis06].

This means that any point lying inside an ϵ -**tube** around the prediction is not penalized, as in Figure 17.19.

The corresponding objective function is usually written in the following form

$$J = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N L_{\epsilon}(\tilde{y}_n, \hat{y}_n) \quad (17.111)$$

where $\hat{y}_n = f(\mathbf{x}_n) = \mathbf{w}^T \mathbf{x}_n + w_0$ and $C = 1/\lambda$ is a regularization constant. This objective is convex and unconstrained, but not differentiable, because of the absolute value function in the loss term. As in Section 11.4.9, where we discussed the lasso problem, there are several possible algorithms we could use. One popular approach is to formulate the problem as a constrained optimization problem. In particular, we introduce **slack variables** to represent the degree to which each point lies outside the tube:

$$\tilde{y}_n \leq f(\mathbf{x}_n) + \epsilon + \xi_n^+ \quad (17.112)$$

$$\tilde{y}_n \geq f(\mathbf{x}_n) - \epsilon - \xi_n^- \quad (17.113)$$

Given this, we can rewrite the objective as follows:

$$J = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{n=1}^N (\xi_n^+ + \xi_n^-) \quad (17.114)$$

This is a quadratic function of $\mathbf{w}$, and must be minimized subject to the linear constraints in Equations 17.112-17.113, as well as the positivity constraints $\xi_n^+ \geq 0$ and $\xi_n^- \geq 0$. This is a standard quadratic program in $2N + D + 1$ variables.

By forming the Lagrangian and optimizing, as we did above, one can show that the optimal solution has the following form

$$\hat{\mathbf{w}} = \sum_n \alpha_n \mathbf{x}_n \quad (17.115)$$

where $\alpha_n \geq 0$ are the dual variables. (See e.g., [SS02] for details.) Fortunately, the $\boldsymbol{\alpha}$ vector is sparse, meaning that many of its entries are equal to 0. This is because the loss doesn't care about errors which are smaller than ϵ . The degree of sparsity is controlled by C and ϵ .

The $\mathbf{x}_n$ for which $\alpha_n > 0$ are called the **support vectors**; these are points for which the errors lie on or outside the ϵ tube. These are the only training examples we need to keep for prediction at test time, since

$$f(\mathbf{x}) = \hat{w}_0 + \hat{\mathbf{w}}^\top \mathbf{x} = \hat{w}_0 + \sum_{n:\alpha_n>0} \alpha_n \mathbf{x}_n^\top \mathbf{x} \quad (17.116)$$

Finally, we can use the kernel trick to get

$$f(\mathbf{x}) = \hat{w}_0 + \sum_{n:\alpha_n>0} \alpha_n \mathcal{K}(\mathbf{x}_n, \mathbf{x}) \quad (17.117)$$

This overall technique is called **support vector machine regression** or **SVM regression** for short, and was first proposed in [VGS97].

In Figure 17.20, we give an example where we use an RBF kernel with $\gamma = 1$. When C is small, the model is heavily regularized; when C is large, the model is less regularized and can fit the data better. We also see that when ϵ is small, the tube is smaller, so there are more support vectors.

17.4 Sparse vector machines

GPs are very flexible models, but incur an $O(N)$ time cost at prediction time, which can be prohibitive. SVMs solve that problem by estimating a sparse weight vector. However, SVMs do not give calibrated probabilistic outputs.

We can get the best of both worlds by using parametric models, where the feature vector is defined using basis functions centered on each of the training points, as follows:

$$\boldsymbol{\phi}(\mathbf{x}) = [\mathcal{K}(\mathbf{x}, \mathbf{x}_1), \dots, \mathcal{K}(\mathbf{x}, \mathbf{x}_N)] \quad (17.118)$$

where $\mathcal{K}$ is any similarity kernel, not necessarily a Mercer kernel. Equation (17.118) maps $\mathbf{x} \in \mathcal{X}$ into $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^N$. We can plug this new feature vector into any discriminative model, such as logistic regression. Since we have $D = N$ parameters, we need to use some kind of regularization, to prevent overfitting. If we fit such a model using ℓ_2 regularization (which we will call **L2VM**, for ℓ_2 -vector machine), the result often has good predictive performance, but the weight vector $\mathbf{w}$ will be dense, and will depend on all N training points. A natural solution is to impose a sparsity-promoting prior on $\mathbf{w}$, so that not all the exemplars need to be kept. We call such methods **sparse vector machines**.

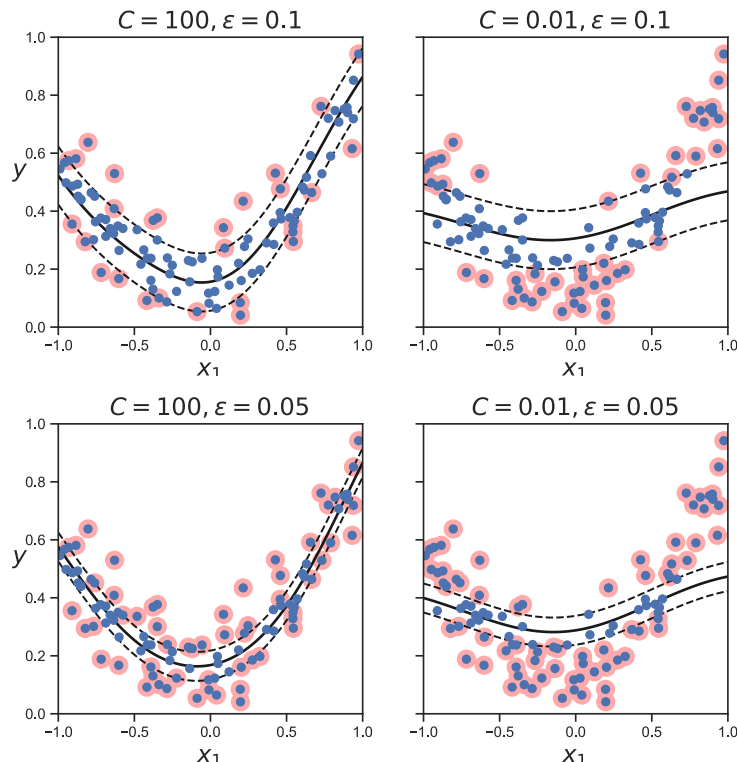


Figure 17.20: Illustration of support vector regression. Adapted from Figure 5.11 of [Gér19]. Generated by `svm_regression_1d.ipynb`.

17.4.1 Relevance vector machines (RVMs)

The simplest way to ensure $\mathbf{w}$ is sparse is to use ℓ_1 regularization, as in Section 11.4. We call this **L1VM** or **Laplace vector machine**, since this approach is equivalent to using MAP estimation with a Laplace prior for $\mathbf{w}$.

However, sometimes ℓ_1 regularization does not result in a sufficient level of sparsity for a given level of accuracy. An alternative approach is based on the use of **ARD** or **automatic relevancy determination**, which uses type II maximum likelihood (aka empirical Bayes) to estimate a sparse weight vector [Mac95; Nea96]. If we apply this technique to a feature vector defined in terms of kernels, as in Equation (17.118), we get a method called the **relevance vector machine** or **RVM** [Tip01; TF03].

17.4.2 Comparison of sparse and dense kernel methods

In Figure 17.21, we compare L2VM, L1VM, RVM and an SVM using an RBF kernel on a binary classification problem in 2d. We use cross validation to pick $C = 1/\lambda$ for the SVM (see Section 17.3.8),

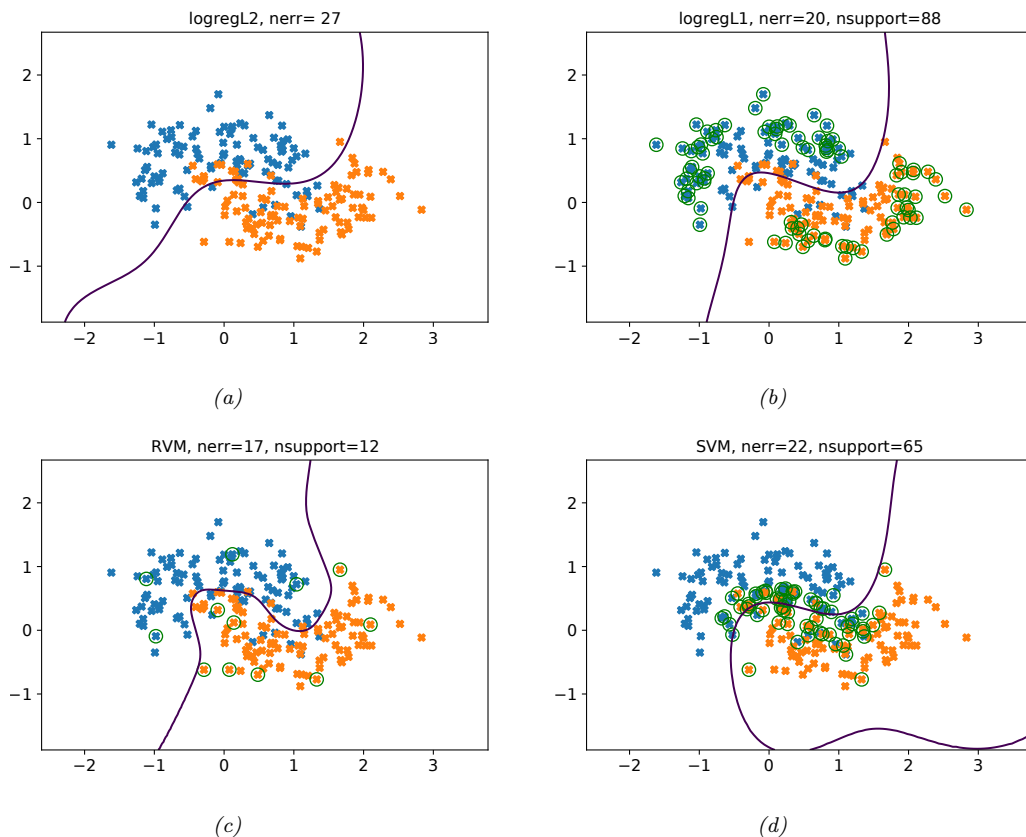


Figure 17.21: Example of non-linear binary classification using an RBF kernel with bandwidth $\sigma = 0.3$. (a) L2VM. (b) L1VM. (c) RVM. (d) SVM. Green circles denote the support vectors. Generated by [kernelBinaryClassifDemo.ipynb](#).

and then use the same value of the regularizer for L2VM and L1VM. We see that all the methods give similar predictive performance. However, we see that the RVM is the sparsest model, so it will be the fastest at run time.

In Figure 17.22, we compare L2VM, L1VM, RVM and an SVM using an RBF kernel on a 1d regression problem. Again, we see that predictions are quite similar, but RVM is the sparsest, then L1VM, then SVM. This is further illustrated in Figure 17.23.

Beyond these small empirical examples, we provide a more general summary of the different methods in Table 17.1. The columns of this table have the following meaning:

- Optimize $\mathbf{w}$: a key question is whether the objective $\mathcal{L}(\mathbf{w}) = -\log p(\mathcal{D}|\mathbf{w}) - \log p(\mathbf{w})$ is convex or not. L2VM, L1VM and SVMs have convex objectives. RVMs do not. GPs are Bayesian methods that integrate out the weights $\mathbf{w}$.

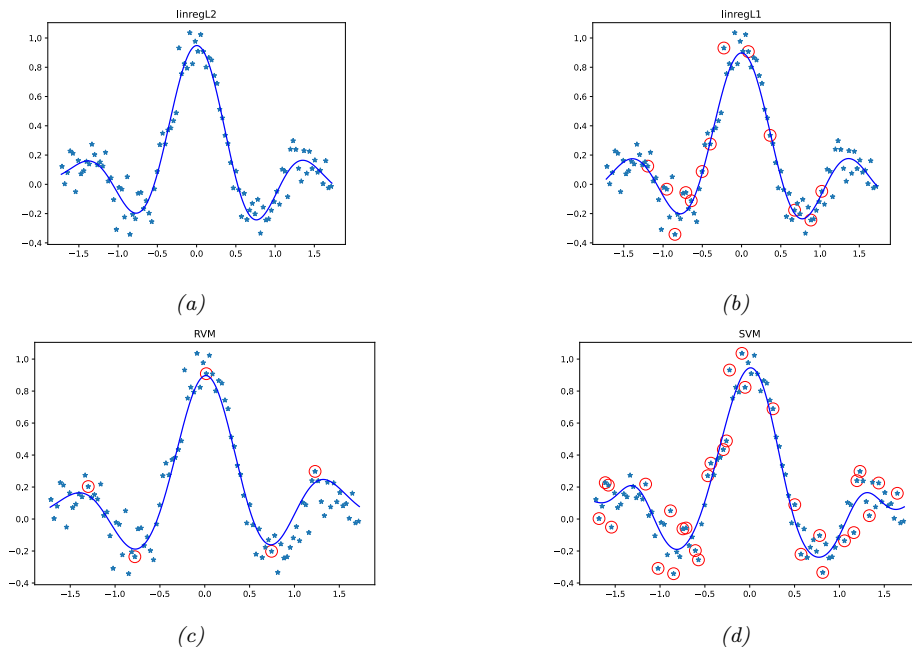


Figure 17.22: Model fits for kernel based regression on the noisy sinc function using an RBF kernel with bandwidth $\sigma = 0.3$. (a) L2VM with $\lambda = 0.5$. (b) L1VM with $\lambda = 0.5$. (c) RVM. (d) SVM regression with $C = 1/\lambda$, chosen by cross validation. Red circles denote the retained training exemplars. Generated by `rvm_regression_1d.ipynb`.

- **Optimize kernel:** all the methods require that we “tune” the kernel parameters, such as the bandwidth of the RBF kernel, as well as the level of regularization. For methods based on Gaussian priors, including L2VM, RVMs and GPs, we can use efficient gradient based optimizers to maximize the marginal likelihood. For SVMs and L1VMs, we must use cross validation, which is slower (see Section 17.3.8).
- **Sparse:** L1VM, RVMs and SVMs are sparse kernel methods, in that they only use a subset of the training examples. GPs and L2VM are not sparse: they use all the training examples. The principle advantage of sparsity is that prediction at test time is usually faster. However, this usually results in overconfidence in the predictions.
- **Probabilistic:** All the methods except for SVMs produce probabilistic output of the form $p(y|\mathbf{x})$. SVMs produce a “confidence” value that can be converted to a probability, but such probabilities are usually very poorly calibrated (see Section 17.3.5).
- **Multiclass:** All the methods except for SVMs naturally work in the multiclass setting, by using a categorical distribution instead of a Bernoulli. The SVM can be made into a multiclass classifier, but there are various difficulties with this approach, as discussed in Section 17.3.7.

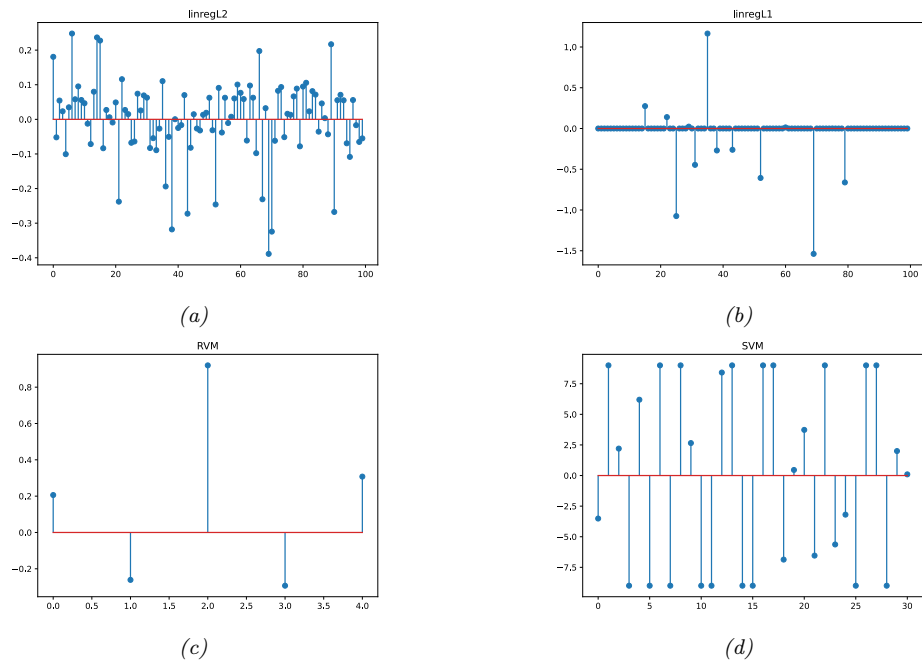


Figure 17.23: Estimated coefficients for the models in Figure 17.22. Generated by `rvm_regression_1d.ipynb`.

Method	Opt. w	Opt. kernel	Sparse	Prob.	Multiclass	Non-Mercer	Section
SVM	Convex	CV	Yes	No	Indirectly	No	17.3
L2VM	Convex	EB	No	Yes	Yes	Yes	17.4.1
L1VM	Convex	CV	Yes	Yes	Yes	Yes	17.4.1
RVM	Not convex	EB	Yes	Yes	Yes	Yes	17.4.1
GP	N/A	EB	No	Yes	Yes	No	17.2.7

Table 17.1: Comparison of various kernel based classifiers. EB = empirical Bayes, CV = cross validation. See text for details.

- Mercer kernel: SVMs and GPs require that the kernel is positive definite; the other techniques do not, since the kernel function in Equation (17.118) can be an arbitrary function of two inputs.

17.5 Exercises

Exercise 17.1 [Fitting an SVM classifier by hand †]

(Source: Jaakkola.) Consider a dataset with 2 points in 1d: $x_1 = 0$ with label $y_1 = -1$ and $x_2 = \sqrt{2}$ with label $y_2 = 1$. Consider mapping each point to 3d using the feature vector $\phi(x) = [1, \sqrt{2}x, x^2]^T$. (This is

equivalent to using a second order polynomial kernel.) The max margin classifier has the form

$$\min ||\mathbf{w}||^2 \quad \text{s.t.} \quad (17.119)$$

$$y_1(\mathbf{w}^T \phi(\mathbf{x}_1) + w_0) \geq 1 \quad (17.120)$$

$$y_2(\mathbf{w}^T \phi(\mathbf{x}_2) + w_0) \geq 1 \quad (17.121)$$

- a. Write down a vector that is parallel to the optimal vector $\mathbf{w}$. Hint: recall from Figure 17.12(a) that $\mathbf{w}$ is perpendicular to the decision boundary between the two points in the 3d feature space.
- b. What is the value of the margin that is achieved by this $\mathbf{w}$? Hint: recall that the margin is the distance from each support vector to the decision boundary. Hint 2: think about the geometry of 2 points in space, with a line separating one from the other.
- c. Solve for $\mathbf{w}$, using the fact that the margin is equal to $1/||\mathbf{w}||$.
- d. Solve for w_0 using your value for $\mathbf{w}$ and Equations 17.119 to 17.121. Hint: the points will be on the decision boundary, so the inequalities will be tight.
- e. Write down the form of the discriminant function $f(x) = w_0 + \mathbf{w}^T \phi(x)$ as an explicit function of x .